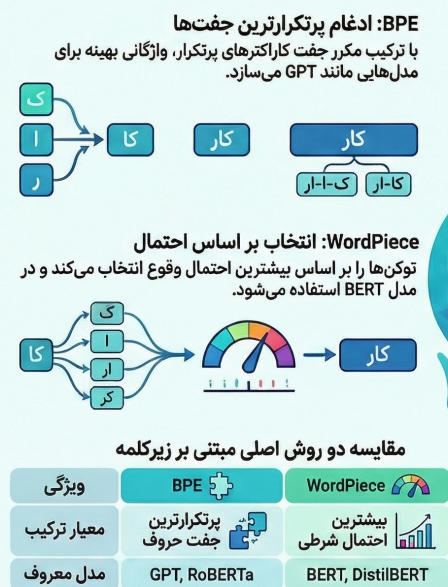
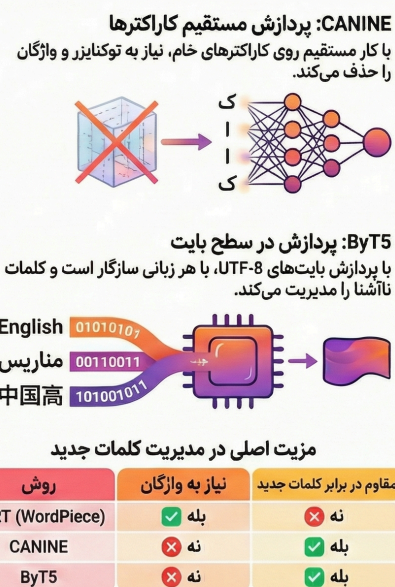


روش‌های توکن‌سازی: زبان ماشین چگونه متن را می‌فهمد؟

رویکرد اول: روش‌های مبتنی بر زیرکلمه (Subword Methods)

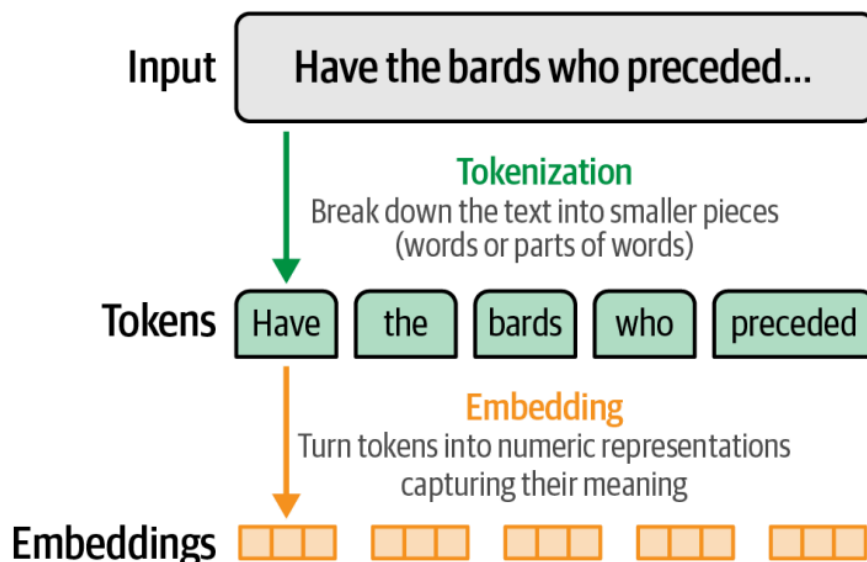


رویکرد دوم: روش‌های بدون واژگان (Vocabulary-Free Methods)



فصل ۲: توکن‌ها و بردارهای تعبیه شده

توکن‌ها (Token) و بردارهای تعبیه شده (Embeddings) دو مفهوم اساسی در استفاده از مدل‌های زبان بزرگ (LLMs) هستند. همان‌طور که در فصل اول دیدیم، این مفاهیم نه تنها برای درک تاریخچه هوش مصنوعی زبان (Language AI) اهمیت دارند، بلکه برای درک نحوه عملکرد، ساختار، و پیشرفت‌های آینده مدل‌های زبان بزرگ ضروری‌اند، همان‌طور که در شکل ۱-۲ مشاهده می‌کنیم.



Input

Have the bards who...

Tokenization

Break down the text into smaller pieces

Tokens

Have

the

b

ards

who

Embeddings

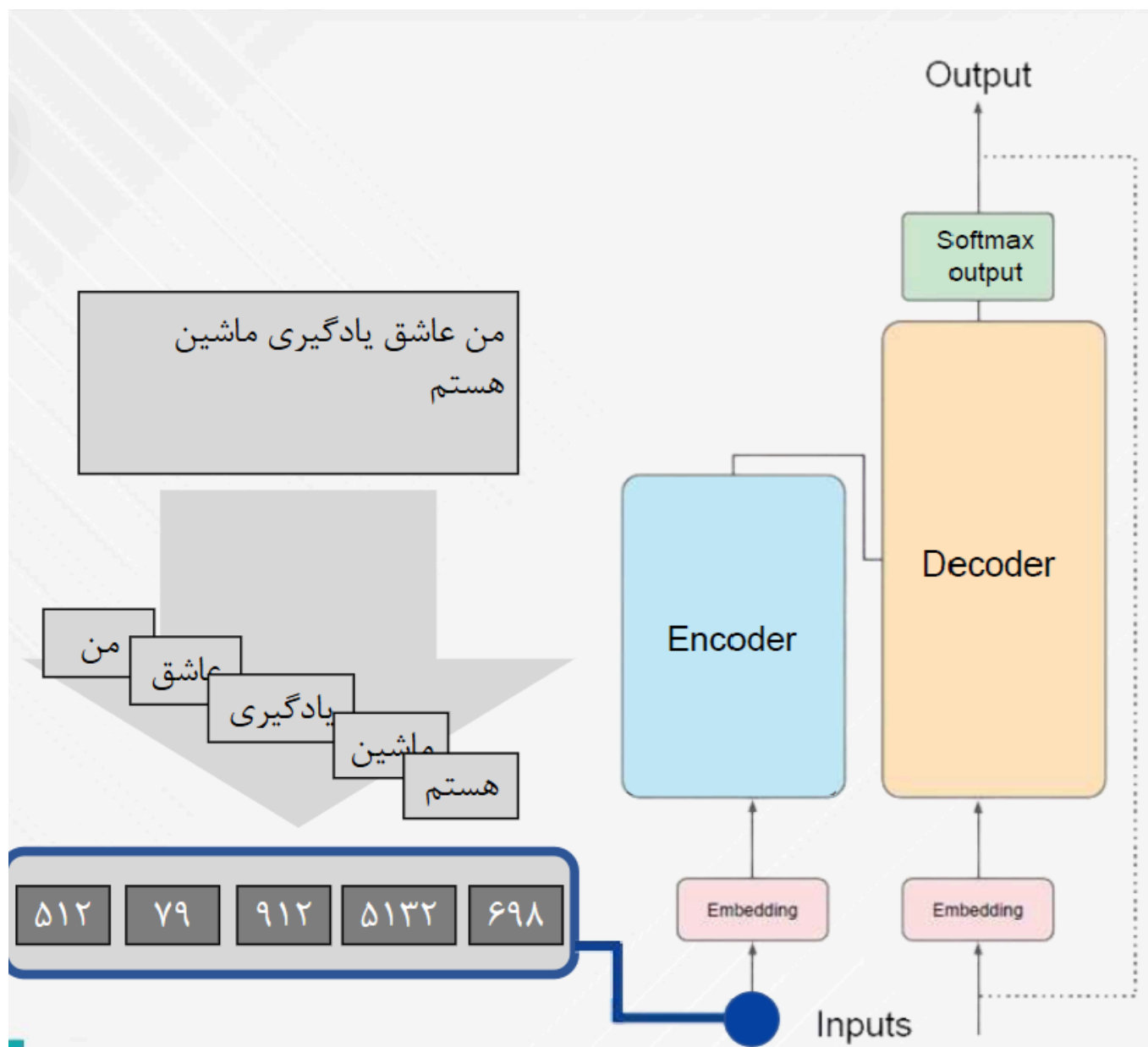
Turn tokens into numeric representations

Embeddings

Language model

Contextualized
Embeddings





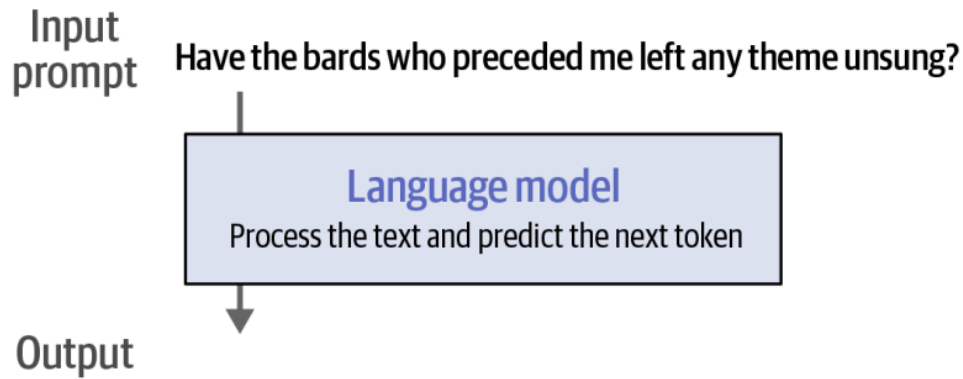
در این فصل با توکن‌ها و روش‌های تبدیل متن به توکن آشنا می‌شویم. سپس روش «word2vec» را بررسی می‌کنیم که قبل از مدل‌های امروزی، برای ساخت نمایش‌های عددی توکن‌ها استفاده می‌شد و به ساخت سیستم‌های توصیه‌گر کمک کرد. در نهایت، یاد می‌گیریم چگونه کل جملات یا متن‌ها را با یک بردار نمایش دهیم تا بتوانیم جستجوی معنایی و مدل‌سازی موضوعات را انجام دهیم، موضوعاتی که در بخش دوم کتاب مطرح می‌شوند.

توکن‌سازی در مدل‌های زبانی بزرگ

تا زمان نگارش این متن، بیشتر افراد از طریق یک محیط وب با مدل‌های زبانی تعامل می‌کنند؛ محیطی که به شکل یک چت بین کاربر و مدل طراحی شده است. ممکن است متوجه شده باشید که مدل پاسخ خود را به صورت یک‌باره نمی‌دهد، بلکه به تدریج و هر بار یک توکن تولید می‌کند. توکن‌ها تنها خروجی مدل نیستند، بلکه همان شکلی هستند که مدل ورودی‌ها را نیز مشاهده می‌کند. متنی که به مدل فرستاده می‌شود، ابتدا به توکن‌ها تقسیم می‌شود، موضوعی که در ادامه بیشتر به آن می‌پردازیم.

چگونه توکنایزرها ورودی‌ها را برای مدل زبانی آماده می‌کنند

مدل‌های زبان مولد یک درخواست ورودی را دریافت کرده و پاسخی را تولید می‌کنند، همان‌طور که در شکل ۲-۲ مشاهده می‌کنیم.



قبل از اینکه پرامپت به مدل زبانی داده شود، ابتدا باید از یک توکنایزر عبور کند که آن را به بخش‌های کوچک‌تر تقسیم می‌کند. مثالی از توکنایزر GPT-4 را می‌توانید در پلتفرم OpenAI مشاهده کنید. اگر متن ورودی را به آن بدهیم، خروجی مانند شکل زیر خواهد بود که در آن هر توکن با رنگ متفاوتی نمایش داده شده است.

GPT-3.5 & GPT-4 GPT-3 (Legacy)

Have the bards who preceded me left any theme unsung?

Clear

Show example

Tokens

13

Characters

53

Have the bards who preceded me left any theme unsung?

Text

Token IDs

دانلود و اجرای مدل زبان بزرگ (LLM)

بیایید با بارگذاری مدل و توکنایزر آن شروع کنیم، مشابه کاری که در فصل اول انجام دادیم:

```

from transformers import AutoModelForCausalLM, AutoTokenizer

# Load model and tokenizer
model = AutoModelForCausalLM.from_pretrained(
    "microsoft/Phi-3-mini-4k-instruct",
    device_map="cuda",
    torch_dtype="auto",
    trust_remote_code=True,
)
tokenizer = AutoTokenizer.from_pretrained("microsoft/Phi-3-mini-4k-instruct")

```

توکنايز کردن مدل زبان بزرگ

حال می‌توانیم خروجی را تولید کنیم. برای این منظور ابتدا درخواست ورودی خود را تعريف می‌کنیم، سپس آن را توکنايز کرده و این توکن‌ها را به مدل ارسال می‌کنیم تا خروجی آن تولید شود. در این مثال، از مدل می‌خواهیم که فقط 20 توکن جديد تولید کند:

```

prompt = "Write an email apologizing to Sarah for the tragic gardening mishap. Explain how it happened.<|assistant|>"
# Tokenize the input prompt
input_ids = tokenizer(prompt, return_tensors="pt").input_ids.to("cuda")

# Generate the text
generation_output = model.generate(
    input_ids=input_ids, max_new_tokens=20 )

# Print the output
print(tokenizer.decode(generation_output[0]))

```

خروجی:

```

<s> Write an email apologizing to Sarah for the tragic gardening mishap. Explain how it happened.<|assistant|> Subject: My Sincere Apologies for the Gardening Mishap

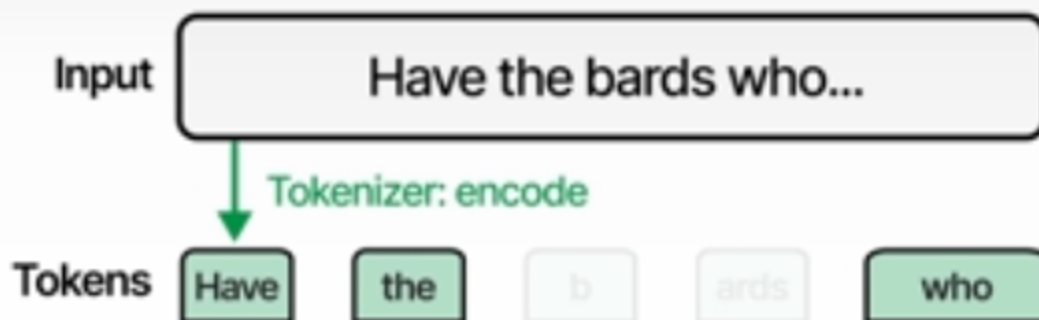
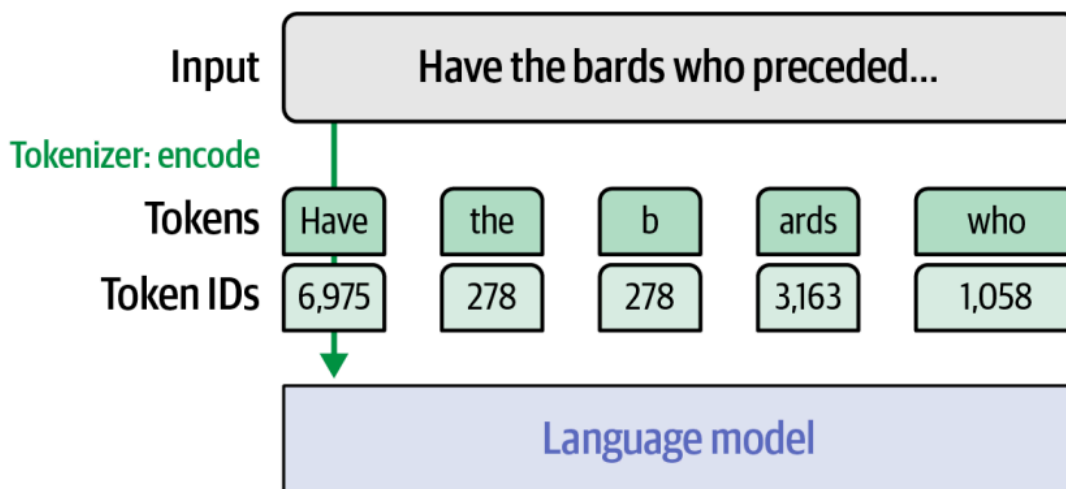
```

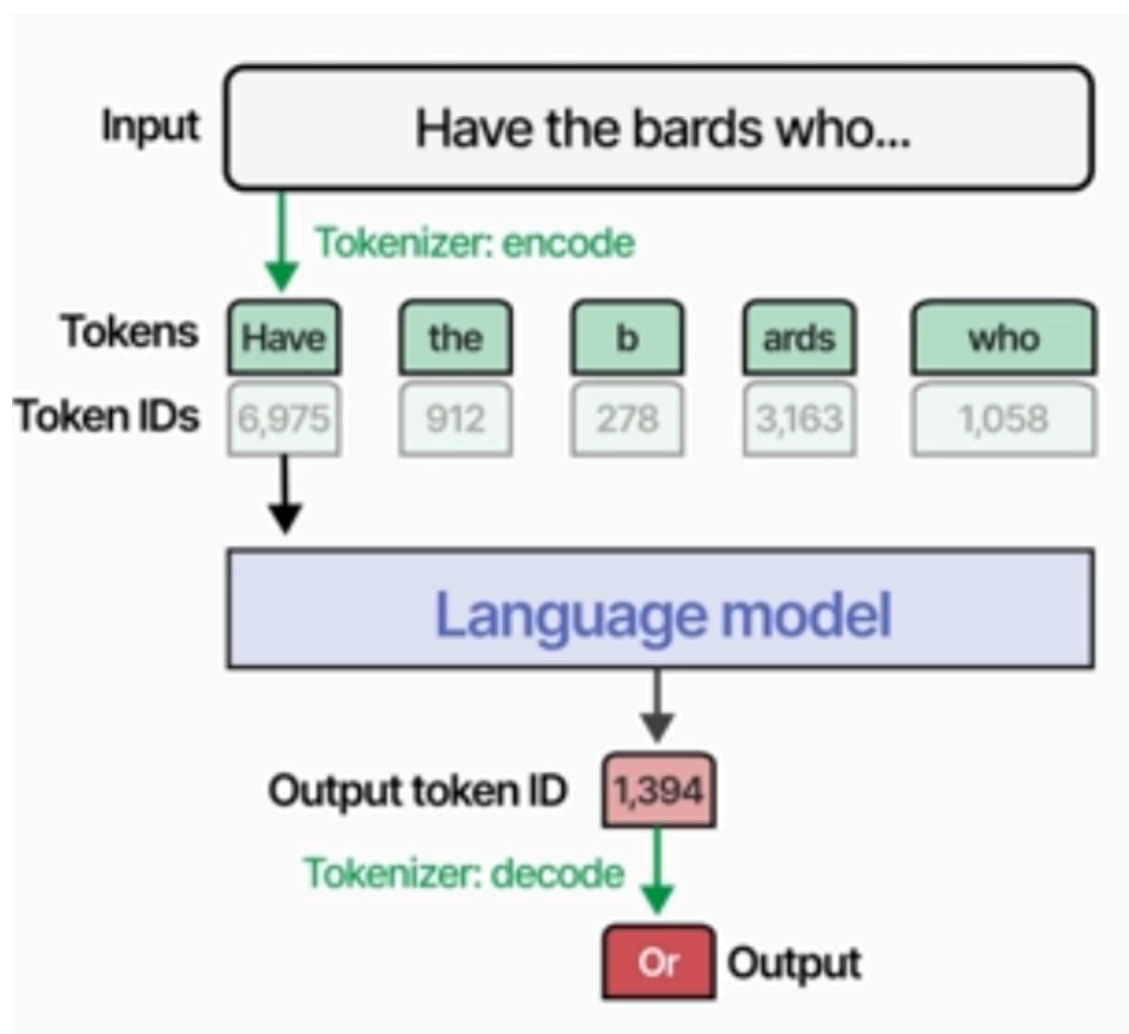
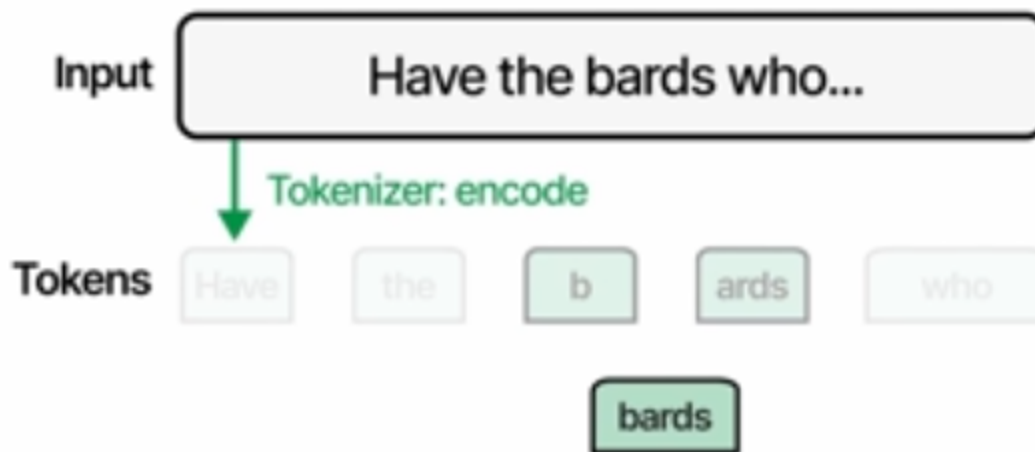
Dear

متنی که به صورت **پررنگ** نمایش داده شده است، همان ۲۰ توکنی است که مدل تولید کرده است. با نگاه به کد، متوجه می‌شویم که مدل در واقع خود متن پرامپت را دریافت نمی‌کند؛ بلکه توکنايزرها ابتدا ورودی را پردازش کرده و اطلاعات مورد نیاز مدل را در متغیر **input_ids** قرار می‌دهند که مدل از آن به عنوان ورودی استفاده می‌کند. بیايد متغیر **input_ids** را چاپ کنیم تا ببینیم چه مقادیری درون آن ذخيره شده است:

```
tensor([[ 1, 14350, 385, 4876, 27746, 5281, 304, 19235, 363, 278, 25305, 293,
16423, 292, 286, 728, 481, 29889, 12027, 7420, 920, 372, 9559, 29889, 32001]],
device='cuda:0')
```

این مقادیر نشان‌دهنده ورودی‌هایی هستند که مدل‌های زبان بزرگ (LLM) به آنها پاسخ می‌دهند، یعنی یک سری اعداد صحیح که در شکل زیر نشان داده شده است. هر عدد، شناسه‌ی یکتایی است که به یک توکن خاص (یک کاراکتر، کلمه یا بخشی از کلمه) اختصاص دارد. این شناسه‌ها به جدول داخل توکنایزر اشاره می‌کنند که شامل همه توکن‌هایی است که توکنایزر می‌شناسد.





اگر بخواهیم این شناسه‌ها را بررسی کنیم، می‌توانیم از متد `decode` توکنایزر استفاده کنیم تا آن‌ها را دوباره به متنی قابل خواندن تبدیل کنیم:

```
for id in input_ids[0]:  
    print(tokenizer.decode(id))
```

این کد خروجی را به گونه‌ای چاپ می‌کند که هر توکن در یک خط جداگانه نمایش داده می‌شود.

```
<s>
Write
an
email
apolog
izing
to
Sarah
for
the
trag
ic
garden
ing
m
ish
```

```
ap
.
Exp
lain
how
it
happened
.
<|assistant|>
```

در اینجا می‌بینیم که توکنایزر چگونه پرامپت ورودی ما را به اجزای کوچک‌تر (توکن‌ها) تبدیل کرده است. به نکات زیر توجه کنید:

- اولین توکن با شناسه ۱ (<s>)، یک توکن ویژه است که آغاز متن را مشخص می‌کند.
- برخی توکن‌ها کلمات کامل هستند، مانند *an*، *Write* و *email*.
- برخی دیگر تنها بخش‌هایی از یک کلمه را تشکیل می‌دهند، مانند *trag*، *izing*، *apolog* و *ic*.
- علامت‌های نگارشی نیز هرکدام یک توکن مستقل به شمار می‌روند.

نکته‌ی جالب توجه این است که **کاراکتر فاصله (space)** به‌تنهایی یک توکن مجزا نیست. در عوض، توکن‌هایی که بخشی از یک کلمه هستند (مانند "izing" یا "ic")، در ابتدای خود یک **کاراکتر پنهان ویژه** دارند که مشخص می‌کند این بخش به کلمه‌ی قبلی متصل است. اگر توکنی فاقد این کاراکتر باشد، فرض می‌شود که پیش از آن یک فاصله وجود دارد.

در بخش خروجی مدل نیز می‌توانیم توکن‌هایی را که تولید شده‌اند بررسی کنیم. با چاپ مقدار متغیر `generation_output`، هم توکن‌های ورودی و هم توکن‌های تولیدشده را مشاهده خواهیم کرد. توکن‌های خروجی (جدید) در این نمایش به‌صورت **پررنگ** مشخص شده‌اند.

```
tensor([[ 1, 14350, 385, 4876, 27746, 5281, 304, 19235, 363, 278,
25305, 293, 16423, 292, 286, 728, 481, 29889, 12027, 7420,
920, 372, 9559, 29889, 32001, 3323, 622, 29901, 1619, 317,
3742, 406, 6225, 11763, 363, 278, 19906, 292, 341, 728,
481, 13, 13, 29928, 799]], device='cuda:0')
```

در اینجا می‌بینیم که مدل ابتدا توکن **۳۳۲۳** را تولید کرده که معادل 'Sub' است، و بلافاصله پس از آن، توکن **۶۲۲** با مقدار 'ject' آمده است. این دو توکن در کنار هم، واژه‌ی **'Subject'** را تشکیل داده‌اند. سپس توکن **۲۹۹۰۱** تولید شده که معادل کاراکتر دونقطه ':' است... و این روند ادامه دارد.

همان‌طور که در بخش ورودی نیاز داشتیم توکن‌ها را به متن قابل‌خواندن تبدیل کنیم، در بخش خروجی نیز به همین ابزار نیاز داریم. برای این کار، از متد `decode` توکنایزر استفاده می‌کنیم. این متد می‌تواند یک **توکن منفرد** یا یک **فهرست از توکن‌ها** را به متن تبدیل کند.

```
print(tokenizer.decode(3323))
print(tokenizer.decode(622))
print(tokenizer.decode([3323, 622]))
print(tokenizer.decode(29901))
```

این دستور خروجی زیر را تولید می‌کند:

```
Sub
ject
Subject
:
```

توکنایزر چگونه متن را به توکن تبدیل می‌کند؟

نحوه تبدیل یک متن به توکن‌ها توسط توکنایزر، تحت تأثیر سه عامل اصلی قرار دارد:

۱. انتخاب روش توکن‌سازی در مرحله طراحی مدل:

نخستین تصمیم مهم، انتخاب روش توکن‌سازی است که در زمان طراحی مدل انجام می‌شود. دو روش پرکاربرد عبارت‌اند از **Byte Pair Encoding (BPE)** که در بسیاری از مدل‌های GPT استفاده می‌شود و

WordPiece که در BERT به کار رفته است. این روش‌ها هدفی مشابه دارند—یعنی ساخت مجموعه‌ای بهینه از توکن‌ها برای نمایش متن‌ها—اما فرآیند دستیابی به این هدف در آن‌ها متفاوت است.

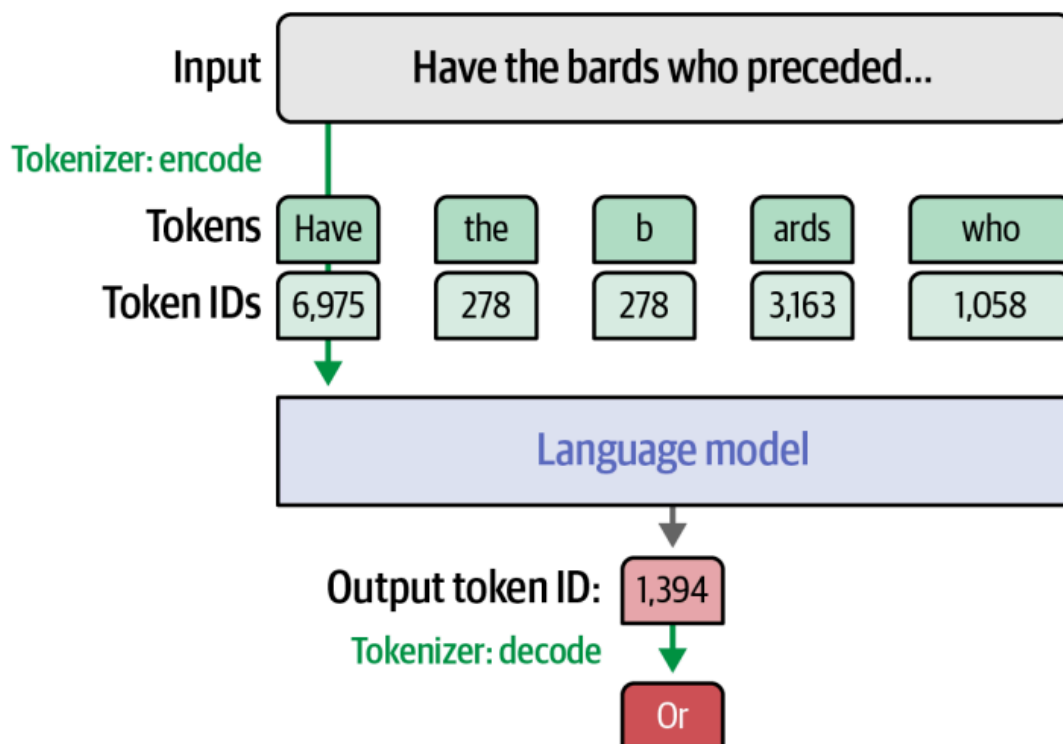
2. تعیین تنظیمات توکنایزر:

پس از انتخاب روش، نوبت به تصمیم‌گیری دربارهٔ تنظیماتی مانند **اندازه واژگان (vocabulary size)** و **تعریف توکن‌های ویژه** مانند `<s>` یا `[CLS]` می‌رسد. این تنظیمات نقش مهمی در عملکرد نهایی توکنایزر دارند. جزئیات بیشتر درباره تفاوت این تنظیمات را در بخش "مقایسه توکنایزرهای آموزش‌دیده" می‌توانید مطالعه کنید.

3. آموزش توکنایزر روی داده‌های متنی خاص:

توکنایزر باید روی یک مجموعه داده مشخص آموزش ببیند تا واژگانی مناسب و کارآمد برای همان نوع داده استخراج کند. حتی اگر از روش و تنظیمات یکسانی استفاده کنیم، توکنایزری که بر روی متون انگلیسی آموزش دیده است، با توکنایزری که بر داده‌های کد یا متون چندزبانه آموزش دیده، عملکرد متفاوتی خواهد داشت.

در نهایت، توکنایزر تنها برای پردازش ورودی به کار نمی‌رود؛ بلکه در خروجی نیز نقش دارد. زمانی که مدل پاسخ تولید می‌کند، این پاسخ به صورت **شناسه‌های عددی (token IDs)** ارائه می‌شود. توکنایزر با تبدیل این شناسه‌ها به متن قابل فهم، خروجی نهایی را در اختیار کاربر قرار می‌دهد—همان‌طور که در شکل زیر مشاهده می‌شود.



تفاوت بین توکن‌های کلمه‌ای، زیرکلمه‌ای، کاراکتری و بایتی abc

Text Have the 🎵 bards who preceded...

Word tokens

Have	the	🎵	bards	who	preceded	...
------	-----	---	-------	-----	----------	-----

Subword tokens

Have	the	🎵	bard	s	who	preced	ed	...
------	-----	---	------	---	-----	--------	----	-----

Character tokens

H	a	v	e		t	h	e		🎵		b	a	r	d	s	...
---	---	---	---	--	---	---	---	--	---	--	---	---	---	---	---	-----

Byte tokens

H a v e				<space>	t h e				<space>	🎵				<space>	
0	0	0	0	0	0	0	0	0	0	1	1	1	1	0	
1	1	1	1	0	1	1	1	1	0	1	0	0	0	0	
0	1	1	1	1	1	1	1	1	1	1	0	0	1	1	
0	0	1	0	0	1	0	0	0	1	1	0	1	0	0	
1	0	0	0	0	0	1	0	0	0	1	1	0	0	0	...
0	0	1	1	0	1	0	1	0	0	1	1	1	0	0	
0	0	1	0	0	0	0	0	0	0	1	1	0	0	0	
0	1	0	1	0	0	0	1	0	0	1	0	1	0	0	

روشی که پیش‌تر درباره‌اش صحبت کردیم، «توکن‌سازی زیرکلمه‌ای» (Subword Tokenization) نام دارد که امروزه رایج‌ترین روش توکن‌سازی در مدل‌های زبانی است. با این حال، این تنها روش موجود نیست. به‌طور کلی چهار نوع شناخته‌شده از توکن‌سازی وجود دارد:

● توکن‌های کلمه‌ای (Word Tokens)

این روش در الگوریتم‌های قدیمی‌تر مانند Word2Vec پرکاربرد بود، اما امروزه در NLP کمتر استفاده می‌شود. با این حال، هنوز در حوزه‌هایی مانند سیستم‌های پیشنهاددهنده کاربرد دارد.

یکی از چالش‌های مهم این روش، ناتوانی آن در مواجهه با کلمات جدید است؛ چرا که اگر واژه‌ای پس از آموزش توکنایزر ظاهر شود، قابل شناسایی نیست. همچنین در این روش، واژگان شامل تعداد زیادی توکن مشابه و بسیار نزدیک به هم می‌شوند (مثلاً apologize، apology، apologist و apologetic). این مشکل با توکن‌سازی زیرکلمه‌ای تا حد زیادی حل شده است، چرا که مثلاً برای «apolog» یک توکن اصلی تعریف می‌شود و باقی‌مانده کلمه‌ها (مثل -ist، -etic، -ize، -y) به‌عنوان پسوندهای قابل ترکیب در واژگان گنجانده می‌شوند.

● توکن‌های زیرکلمه‌ای (Subword Tokens)

در این روش، هم کلمات کامل و هم بخش‌هایی از آن‌ها به‌عنوان توکن در نظر گرفته می‌شوند. از مزایای مهم آن می‌توان به دو مورد اشاره کرد:

1. واژگان فشرده‌تر و بیان‌پذیرتر که ترکیب‌های متعددی را پوشش می‌دهند.
2. توانایی تجزیه کلمات ناشناخته به قطعات کوچکتر که اغلب در واژگان مدل یافت می‌شوند.

● توکن‌های کاراکتری (Character Tokens)

در این روش، هر حرف به‌تنهایی یک توکن محسوب می‌شود. مزیت آن در پوشش کامل واژه‌های جدید است، چرا که همیشه می‌توان واژه‌ها را با حروف اصلی ساخت. اما این کار مدل‌سازی را دشوارتر می‌کند. مثلاً به‌جای استفاده از یک توکن برای «play»، مدل باید حروف p، l، a، y را جداگانه درک و مدل‌سازی کند.

از سوی دیگر، مدل‌هایی که از توکن‌سازی زیرکلمه‌ای استفاده می‌کنند می‌توانند متن بیشتری را در یک پنجره محدود (context window) جای دهند. به‌طور معمول، توکن‌های زیرکلمه‌ای میانگین سه حرف دارند؛ بنابراین در یک پنجره ۱۰۲۴ توکنی، می‌توان حدود سه برابر متن بیشتری نسبت به روش کاراکتری جای داد.

● توکن‌های بایتی (Byte Tokens)

در این روش، توکن‌ها بر اساس بایت‌های تشکیل‌دهنده کاراکترهای یونیکد ساخته می‌شوند. این شیوه را می‌توان در مقاله‌هایی مانند **CANINE** (یک مدل بدون توکنایزر) و **ByT5** (مدلی بایت‌محور برای زبان‌های گوناگون) مشاهده کرد.

نکته‌ای که باید توجه داشت این است که برخی توکنایزرهای زیرکلمه‌ای (مانند GPT-2 و RoBERTa) در مواقع خاص از بایت‌ها به‌عنوان آخرین گزینه برای پوشش کاراکترهای ناشناخته استفاده می‌کنند. با این حال، این‌ها مدل‌های تماماً بایت‌محور نیستند، چرا که از بایت‌ها فقط برای بخش محدودی از توکن‌ها بهره می‌برند، نه کل متن.

نتیجه‌گیری

در مجموع، انتخاب نوع توکن‌سازی تأثیر زیادی بر عملکرد مدل دارد. اگر مایل به بررسی دقیق‌تر این موضوع هستید، مباحث پیشرفته‌تر در کتاب *Designing Large Language Model Applications* مطرح شده‌اند.

روش کلی کار با توکنایزر

```
In [ ]: from transformers import AutoTokenizer
```

```
In [ ]: # define the sentence to tokenize  
sentence = "Hello world!"
```

```
In [ ]: # Load the pretrained tokenizer  
tokenizer =  
    AutoTokenizer.from_pretrained("bert-base-cased")
```

You'll now apply the tokenizer to the sentence. The tokenizer splits the sentence into tokens and returns the IDs of each token.

```
In [ ]: # apply the tokenizer to the sentence and extract the token ids  
token_ids = tokenizer(sentence).input_ids
```

```
In [ ]: print(token_ids)
```

To map each token ID to its corresponding token, you can use the `decode` method of the tokenizer.

```
In [ ]: for id in token_ids:  
    print(tokenizer.decode(id))
```

مقایسه توکنایزهای آموزش دیده در مدل های زبانی بزرگ (LLM) 🔍

پیش تر گفتیم که سه عامل اصلی، نوع توکن هایی را که یک توکنایزر تولید می کند، تعیین می کنند:

1. روش توکن سازی (مثل BPE یا WordPiece)
2. پارامترها و توکن های ویژه که هنگام ساخت توکنایزر تعیین می شوند
3. داده هایی که توکنایزر روی آن آموزش دیده است

در این بخش، می خواهیم چند توکنایزر واقعی و آموزش دیده را با هم مقایسه کنیم تا ببینیم چگونه انتخاب های مختلف، رفتار آن ها را تغییر می دهد. همچنین خواهیم دید که مدل های تخصصی (مثل مدل های تولید کد) معمولاً به توکنایزهای مخصوص خود نیاز دارند.

متن آزمایشی برای توکن سازی 🧪

برای مقایسه توکنایزهای مختلف، از متن زیر استفاده می کنیم که انواع مختلفی از ورودی ها را پوشش می دهد:

```
text = ""  
English and CAPITALIZATION  
🐦
```

```
show_tokens False None elif == >= else: two tabs:" " Three tabs: " "
12.0*50=600
"""
```

این متن موارد متنوعی را در بر می‌گیرد:

- ✓ حروف بزرگ و کوچک (مانند CAPITALIZATION)
- 🌐 زبان‌های غیر انگلیسی (مثل حرف چینی 𑀓)
- 🎵 ایموجی‌ها (مثل 🎵)
- 🧑💻 کدهای برنامه‌نویسی (مثلاً elif , == , <=)
- 🧮 اعداد و عملیات ریاضی
- ♦ توکن‌های ویژه (مثل [CLS] ، [SEP] و ... که برای علامت‌گذاری شروع/پایان متن به کار می‌روند)

📦 از توکنایزهای قدیمی تا جدید

می‌خواهیم ببینیم:

- هر توکنایزر چگونه این متن را به توکن‌ها می‌شکند
- خروجی آن چه تفاوت‌هایی با دیگر توکنایزرها دارد
- چه تأثیری بر عملکرد مدل دارد

🖍 تابع رنگی برای نمایش توکن‌ها

برای نمایش رنگی توکن‌های تولید شده توسط هر توکنایزر، از کد زیر استفاده می‌کنیم:

```
from transformers import AutoTokenizer

colors_list = [
    '102;194;165', '252;141;98', '141;160;203',
    '231;138;195', '166;216;84', '255;217;47'
]

def show_tokens(sentence, tokenizer_name):
    tokenizer = AutoTokenizer.from_pretrained(tokenizer_name)
    token_ids = tokenizer(sentence).input_ids
    for idx, t in enumerate(token_ids):
        print(
            f'\x1b[0;30;48;2;{colors_list[idx % len(colors_list)]}m' +
            tokenizer.decode(t) +
            '\x1b[0m',
```

```
end=' '
)
```

مثال: اجرای تابع برای چند مدل مختلف

توکنایزهای پیشنهادی برای مقایسه:

مدل	نوع توکن سازی	توضیح
bert-base-uncased	WordPiece	کلاسیک، مناسب برای زبان انگلیسی
gpt2	Byte-Pair Encoding	توکنایزر بایتی با fallback برای حروف
google/byt5-small	Byte-level	مدل بدون توکنایزر کلاسیک (token-free)
bigcode/starcoder	Code-specialized BPE	توکنایزر ویژه‌ی تولید کد (مثل Python)

مثال اجرا:

```
show_tokens(text, "bert-base-uncased")
show_tokens(text, "gpt2")
show_tokens(text, "google/byt5-small")
show_tokens(text, "bigcode/starcoder")
```

نکته آموزشی مهم

با مقایسه خروجی این توکنایزرها، متوجه می‌شویم که:

- برخی توکنایزرها با ایموجی یا زبان چینی مشکل دارند (مثلاً WordPiece)
- برخی دیگر، مانند GPT-2 یا ByT5، انعطاف‌پذیری بیشتری دارند
- مدل‌های تولید کد، فضای خالی (indentation) یا کلیدواژه‌ها را دقیق‌تر تشخیص می‌دهند

مدل پایه BERT (نسخه Uncased - سال ۲۰۱۸)

 [لینک به مدل در Hugging Face](#)

مشخصات کلیدی 

- روش توکن‌سازی: WordPiece (روشی که اولین بار در جستجوی صوتی ژاپنی و کره‌ای گوگل معرفی شد)
- حجم واژگان: ۳۰,۵۲۲ توکن
- حساسیت به حروف بزرگ/کوچک: ✖ خیر (uncased) همه حروف بزرگ به کوچک تبدیل می‌شوند.

♦ توکن‌های ویژه (Special Tokens)

توکن	نقش
[CLS]	توکن طبقه‌بندی؛ همیشه در ابتدای ورودی می‌آید.
[SEP]	جداکننده بین دو جمله یا بخش؛ مناسب برای کارهای چندبخشی (مثل reranking).
[PAD]	توکن پرکننده (padding) برای یکنواخت‌سازی طول ورودی‌ها.
[MASK]	برای مخفی‌سازی کلمات در آموزش (Masked LM).
[UNK]	برای نمایش توکن‌هایی که در واژگان مدل وجود ندارند (unknown token).

abc خروجی توکن‌شده برای متن نمونه:

[CLS] english and capital ##ization [UNK] [UNK] show token ##s false none
eli ##f = = > = else : two tab ##s : " " three tab ##s : " " 12 . 0 * 50 = 600 [SEP]

🔍 نکات مهم در مورد رفتار توکنایزر این مدل

1. خطاهای جدید (newline) حذف شده‌اند
➤ یعنی مدل تفاوتی بین خطوط قائل نیست و ممکن است ساختار متنی مانند چت یا شعر را درک نکند.
2. تمام حروف به کوچک تبدیل شده‌اند
➤ مثلاً CAPITALIZATION تبدیل شده به capital ##ization.
3. شکستن کلمات به زیرکلمه (Subword Splitting)
➤ مثلاً:
capitalization ← capital, ##ization
علامت ## یعنی این زیرکلمه باید به کلمه قبلی بچسبد.
4. توکن‌های ناشناخته (مثل ایموجی یا کاراکتر چینی)
➤ حذف شده و با [UNK] جایگزین شده‌اند، چون در واژگان WordPiece نبودند.

نتیجه‌گیری

مدل BERT پایه‌ی Uncased با WordPiece:

- در زبان انگلیسی بسیار قدرتمند است
- اما در تشخیص زبان‌های غیرلاتین (مثل فارسی یا چینی) و نشانه‌هایی مثل ایموجی‌ها محدودیت دارد
- از طریق زیرکلمه‌ها می‌تواند واژگان جدید را تا حدی مدیریت کند

مدل پایه BERT (نسخه Cased - سال ۲۰۱۸)

مشاهده در [Hugging Face](#)

مشخصات فنی

ویژگی	مقدار
روش توکن‌سازی	WordPiece
سایز واژگان	28,996 توکن
حساسیت به حروف بزرگ	بله (برخلاف نسخه uncased) ✓

توکن‌های ویژه (مثل نسخه‌ی uncased)

توکن	توضیح
[CLS]	نشانه‌ای برای طبقه‌بندی جمله‌ها (در ابتدای ورودی)
[SEP]	جداکننده‌ی جملات (در پایان یا بین جملات)
[PAD]	توکن پرکننده برای ورودی‌های کوتاه‌تر
[MASK]	توکن پوشاننده برای آموزش ماسک‌شده
[UNK]	نمایش توکن‌های ناشناخته (خارج از واژگان)

خروجی توکن‌شده برای متن نمونه:

[CLS] English and CA ##PI ##TA ##L ##I ##Z ##AT ##ION [UNK] [UNK] show token
 ##s F ##als ##e None el ##if = = > = else : two ta ##bs : " " Three ta ##bs : " "
 12 . 0 * 50 = 600 [SEP]

نکات مهم در مورد توکنایزر cased 🔍

1. حساس به حروف بزرگ است

- برخلاف نسخه‌ی uncased، در اینجا کلمات با حروف بزرگ حفظ می‌شوند.
- مثلاً کلمه CAPITALIZATION شکسته شده به ۸ زیرکلمه:

CA, ##PI, ##TA, ##L, ##I, ##Z, ##AT, ##ION

2. استفاده از [CLS] و [SEP]

- در ابتدا و انتهای هر ورودی، [CLS] و [SEP] قرار می‌گیرند.
- کاربردها:
- برای طبقه‌بندی جملات یا متن [CLS]
- برای جدا کردن دو جمله در وظایف دوبخشی مثل تطابق پرسش-پاسخ یا رتبه‌بندی [SEP] (مانند آنچه در فصل ۸ استفاده می‌شود)

3. مناسب کاربردهایی که اطلاعات ظاهری مثل حرف بزرگ مهم است

- مثل تحلیل اسامی خاص، ابتدای جمله‌ها، و متن‌های رسمی یا قانونی

4. کاهش حجم واژگان نسبت به نسخه‌ی uncased

- چون کلمات مشابه ولی با حروف بزرگ/کوچک جداگانه در واژگان ثبت می‌شوند، واژگان باید به‌دقت طراحی شود

vs Cased با Uncased تفاوت نسخه

ویژگی	نسخه Cased	نسخه Uncased
حروف بزرگ حفظ می‌شوند؟	بله ✓	خیر ✗
مناسب برای اسامی خاص؟	بله ✓	خیر ✗
حجم واژگان	کمتر (28,996)	بیشتر (30,522)
استفاده از [UNK] برای ایموجی/حروف غیراستاندارد	بله	بله

مدل **BERT base cased** زمانی توصیه می‌شود که:

- متن شما شامل اطلاعات معنایی وابسته به حروف بزرگ است (مثل "Apple" vs "apple")
 - یا در زبان‌هایی مثل انگلیسی، اسامی خاص، ابتدای جمله‌ها و شکل نوشتار اهمیت دارد
- اما اگر با زبان فارسی، محتوای غیررسمی یا متن‌هایی بدون حساسیت به حروف کار می‌کنید، نسخه‌ی **uncased** یا مدل‌های فارسی خاص مناسب‌ترند.

🤖 مدل GPT-2 (انتشار ۲۰۱۹)

🔗 [لینک به مدل در Hugging Face](#)

✓ مشخصات کلیدی

ویژگی	مقدار
روش توکن‌سازی	Byte Pair Encoding (BPE)
حجم واژگان	50,257 توکن
توکن ویژه	`<
حساس به حروف بزرگ؟	بله
پشتیبانی از ایموجی؟	بله، با بازسازی از چند توکن

📌 نمونه توکن‌شده از متن:

English and CAP ITAL IZ ATION

🔍 🔍 🔍 🔍 🔍

show _ t o k e n s F a l s e N o n e e l i f == >= e l s e : t w o t a b s : " " T h r e e t a b s : " "

12 . 0 * 50 = 600

🔍 نکات مهم در مورد توکنایزر GPT-2

1. حفظ خطوط جدید (newlines)
مدل ساختار خطی متن (مثل چت یا شعر) را حفظ می‌کند.

2. حساس به حروف بزرگ

مثلاً: CAPITALIZATION به چهار توکن تقسیم شده:

CAP, ITAL, IZ, ATION

3. پشتیبانی از کاراکترهای غیرمتنی (مثل ایموجی یا حروف چینی)

- کاراکترهایی مثل 🎵 یا 🇮🇷 به چند توکن تبدیل می‌شوند.
- اگرچه در خروجی مستقیم چاپی به صورت 💎 دیده می‌شوند، اما می‌توان آن‌ها را بازسازی کرد. مثال:

```
🎵 → # tokenizer.decode([8582, 236, 113])
```

4. مدیریت دقیق فضاهای خالی (whitespaces)

- دو تب (\t) به دو توکن جداگانه تبدیل شده‌اند (توکن ID: 197)
- چهار فاصله (space) با سه توکن (توکن ID: 220) نمایش داده شده‌اند، و فاصله‌ی آخر درون توکن نقل‌قول انتهایی است.

5. اهمیت فضای سفید در کدنویسی

- توکنایزهایی که فاصله‌های پشت سر هم را به یک توکن تبدیل می‌کنند، برای مدل‌های زبانی کدنویسی (مثل Python) مناسب‌ترند.
- این کار باعث می‌شود مدل راحت‌تر سطوح تورفتگی (indentation) را تشخیص دهد و عملکرد بهتری در تولید کد داشته باشد.

جمع‌بندی

ویژگی	GPT-2
پشتیبانی از ایموجی و چندزبان؟	بله ✓
حساس به حروف بزرگ؟	بله ✓
حفظ ساختار خطی متن؟	بله ✓
مناسب برای کد و whitespace دقیق؟	✓ نسبتاً، اما نه مثل مدل‌های تخصصی کدنویسی
روش توکن‌سازی؟	(کارآمد و منعطف) BPE

در مدل‌های زبانی، نویسه‌های فاصله (whitespace characters) مانند فاصله، تب (tab) و رفت‌وبرگشت خط (newline)، نقش مهمی در درک و تولید کد ایفا می‌کنند. به‌ویژه در زبان‌هایی مانند پایتون که ساختار کد وابسته به تورفتگی (indentation) است، نحوه‌ی نمایش این فاصله‌ها اهمیت زیادی دارد. اگر مدل بتواند دنباله‌ای از چهار نویسه فاصله را به‌صورت یک توکن واحد در نظر بگیرد، تطابق بهتری با داده‌های کد پایتون خواهد داشت. در مقابل، اگر این فاصله‌ها به چهار توکن جداگانه تبدیل شوند، مدل باید تورفتگی را به‌صورت دستی دنبال کند که این موضوع می‌تواند فرآیند یادگیری را پیچیده‌تر کرده و عملکرد مدل را کاهش دهد. بنابراین، انتخاب مناسب روش توکنیزه‌سازی می‌تواند تأثیر قابل توجهی بر بهبود توانایی مدل در انجام وظایف خاص، مانند تحلیل یا تولید کد، داشته باشد.

در مدل Flan-T5 (2022)، روش توکنیزاسیون مورد استفاده **SentencePiece** است که از مدل‌های **BPE** و **unigram** پشتیبانی می‌کند. این روش به‌طور خاص برای پردازش متون به‌صورت زبان‌محور طراحی نشده و در انواع زبان‌ها قابل استفاده است.

ویژگی‌های کلیدی توکنیزاسیون در Flan-T5:

- اندازه واژگان: ۳۲,۱۰۰ توکن
- توکن‌های ویژه:
- برای کلمات ناشناخته `<unk>`
- برای پدینگ `<pad>`
- نکات قابل توجه در عملکرد **SentencePiece** در این مدل:
 1. نبود توکن خاص برای نویسه‌های فاصله و خط جدید (**newline**):
 - در توکنیزاسیون Flan-T5، فاصله‌ها و تب‌ها (tab) به شکل توکن مستقل نمایش داده نمی‌شوند. این مسئله، کار مدل را در فهم ساختار کد (به‌خصوص در زبان‌هایی مانند Python که وابسته به تورفتگی هستند) دشوار می‌کند.
 2. مقابله ضعیف با اموجی‌ها و نویسه‌های غیرلاتین (مثلاً چینی):
 - این نوع نویسه‌ها به‌طور پیش‌فرض به `<unk>` نگاشت می‌شوند، به این معنا که مدل در درک یا تولید آن‌ها ناتوان است.
 3. نمونه توکنیزه‌شده متن:

```
English and CA PI TAL IZ ATION <unk> <unk> show _ to ken s Fal s e None e l i f = = >
= else : two tab s : " " Three tab s : " " 12. 0 * 50 = 600 </s>
```

 - جملاتی مانند English and CA PI TAL IZ ATION به توکن‌هایی جدا از هم شکسته می‌شوند.
 - اموجی‌ها و نویسه‌های چینی همگی به `<unk>` تبدیل می‌شوند.
 - ساختارهایی مانند شرط‌های پایتون (`if == > = else`) به‌صورت تکه‌تکه پردازش می‌شوند، و تو رفتگی‌ها (tabها) به شکل عبارات متنی مثل `two tabs` نمایش یافته‌اند، نه توکن واقعی تب.

نتیجه‌گیری:

روش توکنیزاسیون Flan-T5 به دلیل نبود پشتیبانی از نویسه‌های فاصله و **newline**، و همچنین نگاشت نویسه‌های ناشناخته به `<unk>`، باعث کاهش توانایی مدل در درک کدهای برنامه‌نویسی، کار با زبان‌های غیرلاتین، و استفاده از اموجی‌ها می‌شود. این محدودیت‌ها می‌توانند عملکرد مدل را در وظایفی مانند برنامه‌نویسی یا پردازش متون چندزبانه تضعیف کنند. انتخاب روش توکنیزاسیون دقیق و هدفمند می‌تواند در این زمینه تأثیر بسزایی در بهبود کیفیت مدل داشته باشد.

در مدل GPT-4 (عرضه شده در سال 2023)، از روش توکنیزاسیون **BPE (Byte Pair Encoding)** استفاده شده است.

ویژگی‌های کلیدی توکنیزاسیون در GPT-4:

- اندازه واژگان: کمی بیش از ۱۰۰,۰۰۰ توکن
- توکن‌های ویژه:
 - `<|endoftext|>` برای پایان متن
 - سه توکن خاص برای تولید متن در میانه (**fill-in-the-middle**) که به مدل اجازه می‌دهد علاوه بر متن قبل، متن بعد از یک بخش را نیز در نظر بگیرد:
 - `<|fim_prefix|>`: بخش آغازین
 - `<|fim_middle|>`: بخش میانی که باید تولید شود
 - `<|fim_suffix|>`: بخش پایانی متن

این تکنیک به‌طور کامل در مقاله‌ای با عنوان

توضیح داده شده که جزئیات آن "*Efficient training of language models to fill in the middle*" فراتر از بحث این متن است.

نمونه‌ای از متن توکنیزه شده:

English and CAPITAL IZATION



show _tokens False None elif == >= else : two tabs : " " Three tabs : " " "

12 . 0 * 50 = 600

نکات قابل توجه در مورد توکنیزاسیون GPT-4:

1. شناخت توالی‌های فاصله:

- این توکنیزر قادر است دنباله‌ای از چند فاصله پشت سر هم (مثلاً چهار فاصله) را با یک توکن واحد نمایش دهد.
- در واقع، برای توالی‌هایی از فاصله‌ها تا سقف ۸۳ فضای پشت سر هم، توکن خاص دارد.
- این ویژگی برای پردازش کدهای برنامه‌نویسی، که وابسته به تورفتگی هستند، بسیار مفید است.

2. شناخت کلیدواژه‌های زبان برنامه‌نویسی:

- واژه‌ی کلیدی `elif` در زبان پایتون، به‌صورت یک توکن مستقل در GPT-4 وجود دارد.
- این ویژگی نشان می‌دهد که GPT-4 نه‌تنها برای زبان طبیعی، بلکه برای کدنویسی نیز بهینه شده است.

3. کاهش تعداد توکن‌ها برای واژه‌های رایج:

- مدل GPT-4 معمولاً واژه‌ها را با تعداد توکن کمتری نسبت به مدل‌های قبلی نمایش می‌دهد.
- برای مثال:
- "CAPITALIZATION" در GPT-4 → توکن دو
- تنها دو توکن → چهار توکن
- یک توکن → GPT-4 در "tokens"
- در مدل‌های قبلی → سه توکن

4. رفتار مشابه با GPT-2:

- توکنیزاسیون GPT-4 از نظر ساختار پایه به GPT-2 شبیه است، با این تفاوت که دقت و فشردگی بیشتری دارد.
- همچنین، رفتار آن با نویسه‌هایی مانند « ϵ » مشابه GPT-2 است و می‌توانید برای جزئیات بیشتر به بحث مربوط به توکنیزاسیون GPT-2 مراجعه کنید.

جمع‌بندی:

توکنیزاسیون در GPT-4 نسبت به نسخه‌های قبلی، بهینه‌تر، فشرده‌تر و سازگارتر با زبان برنامه‌نویسی شده است. قابلیت‌هایی مانند توکن‌سازی توالی‌های فاصله، شناسایی کلیدواژه‌های برنامه‌نویسی، و استفاده از توکن‌های ویژه برای تولید در میانه‌ی متن، همگی نشان‌دهنده‌ی طراحی دقیق این مدل برای طیف وسیعی از کاربردهای زبانی و محاسباتی است.

مدل StarCoder2 (2024)

مدل **StarCoder2** یک مدل زبانی پیشرفته با ۱۵ میلیارد پارامتر است که به‌طور ویژه برای تولید کد برنامه‌نویسی طراحی شده است. این مدل در مقاله‌ای با عنوان «*StarCoder 2 و Stack v2: نسل آینده*» معرفی شده و ادامه‌دهنده مسیر مدل قبلی یعنی «*StarCoder: May the source be with you!*» است.

♦ روش رمزگذاری متن (Tokenization)

مدل از روش رمزگذاری جفت‌نویسه‌ای (BPE) استفاده می‌کند؛ یعنی ترکیب‌های رایج حروف و نمادها را به صورت توکن‌های جداگانه شناسایی می‌کند.

- تعداد توکن‌ها (واژگان): ۴۹,۱۵۲
- توکن‌های ویژه مهم:
- پایان متن : `<|endoftext|>`
- توکن‌های مخصوص "تکمیل میانی کد" (Fill-in-the-middle):

- `<fim_prefix>`

`<fim\_middle>` -

`<fim\_suffix>` -

`<fim\_pad>` -

English and CAPITAL IZATION

◆◆◆◆◆

show _ tokens False None elif == >= else : two tabs : " " Three tabs : " "

12.0*50=600

◆ درک زمینه (Context) در کدنویسی

در برنامه‌نویسی، فایل‌ها ممکن است به توابع یا کدهایی در فایل‌های دیگر وابسته باشند. بنابراین مدل باید بتواند:

- کدهای مربوط به فایل‌های مختلف یک پروژه را از هم تشخیص دهد
- بین کدهای مربوط به پروژه‌های مختلف تفاوت قائل شود

برای این منظور، StarCoder2 از توکن‌های اختصاصی برای نام فایل و نام مخزن (repository) استفاده می‌کند:

- `<filename>` : نام فایل
- `<reponame>` : نام مخزن
- `<gh_stars>` : تعداد ستاره‌های مخزن در گیت‌هاب

◆ مزایای ایجاد توکن مجزا برای اعداد و فاصله‌ها

- این روش مانند GPT-4، **فاصله‌ها و تب‌ها (whitespace)** را به صورت توکن مستقل رمزگذاری می‌کند.
- برخلاف بسیاری از مدل‌ها، در StarCoder2 **هر رقم عددی به صورت یک توکن مجزا** پردازش می‌شود. مثلاً عدد ۶۰۰ به صورت ۶، ۰، ۰ توکنیزه می‌شود.
- در اغلب مدل‌ها مخصوصاً مدل‌هایی مثل GPT-2، اعداد به صورت ناهماهنگ رمزگذاری می‌شوند:
- عدد ۸۷۰ یک توکن است
- اما عدد ۸۷۱ دو توکن: ۸ و ۷۱

این تفاوت می‌تواند باعث **درک اشتباه مدل از اعداد** شود. اما StarCoder2 با توکنیزه کردن دقیق ارقام، درک بهتری از اعداد و محاسبات ریاضی دارد.

مدل Galactica که در مقاله‌ی

"Galactica: A large language model for science"

معرفی شده است. این مدل با تمرکز ویژه بر **دانش علمی** ساخته شده و بر پایه حجم زیادی از **مقالات علمی**، **منابع مرجع و پایگاه‌های دانشی** آموزش دیده است.

یکی از نقاط قوت Galactica، **توکن‌سازی هوشمند و تخصصی** آن است. این مدل تلاش می‌کند جزئیات دقیق‌تری از زبان علمی را درک کند و برای این منظور، مدل در **ارجاعات علمی**، **استدلال گام‌به‌گام**، **فرمول‌های ریاضی**، **توالی اسیدهای آمینه و DNA**، توکن‌های ویژه‌ای در نظر گرفته است.

روش توکنیزاسیون: رمزنگاری جفت‌نویسه‌ای (BPE)

اندازه واژگان: ۵۰,۰۰۰

توکن‌های ویژه:

- **آغاز توالی:** <s>
- **پرکننده (برای هم اندازه کردن توالی‌ها با تعداد توکن های متفاوت):** <pad>
- **پایان توالی:** </s>
- **توکن برای واژگان ناشناخته:** <unk>

- **توکن‌های ویژه برای ارجاعات:**

در این مدل، ارجاعات درون دو توکن ویژه قرار می‌گیرند:

- [START_REF]
- [END_REF]

یک نمونه از نحوه‌ی استفاده از این توکن‌ها در مقاله به‌صورت زیر است:

Recurrent neural networks, long short-term memory [START_REF] Long Short-Term Memory, Hochreiter [END_REF]

این روش به مدل اجازه می‌دهد تا نقل‌قول‌ها و منابع علمی را بهتر درک و مدیریت کند، که برای پردازش متون علمی و تحقیقاتی بسیار سودمند است.

- **توکن مخصوص استدلال گام‌به‌گام:** مدل از یک توکن ویژه برای استدلال زنجیره‌ای یا تفکر گام‌به‌گام (Chain-of-Thought) استفاده می‌کند:

- <work>

این توکن به مدل کمک می‌کند تا فرایندهای استدلالی را بهتر مدیریت کند و بتواند مراحل مختلف حل یک مسئله یا تحلیل یک متن را به‌صورت پیوسته تولید کند.

استفاده از این توکن در مدل‌های زبانی بزرگ (LLMs)، به‌ویژه در حل مسائل ریاضی، برنامه‌نویسی، و پاسخ به سوالات تحلیلی بسیار مفید است.

English and CAPITALIZATION

⬇️⬇️⬇️⬇️⬇️⬇️

show _ tokens False None elif == > = else : two t abs : " " Three t abs : " "

12.0*50=600

در این مثال‌ها می‌بینیم که:

- حروف بزرگ و کوچک انگلیسی، جداگانه در نظر گرفته می‌شوند.
- اعداد نیز به صورت رقم‌به‌رقم توکنیزه می‌شوند (مثلاً عدد 600 به صورت 0 0 6).

ویژگی‌های توکنیزاسیون:

مدل Galactica، مانند StarCoder2، توجه ویژه‌ای به **کد و فاصله‌ها** دارد. ویژگی‌های توکن‌سازی آن به شکل زیر است:

- **فاصله‌ها و تب‌ها (whitespace & tabs)** به صورت یک توکن واحد رمزنگاری می‌شوند؛ یعنی اگر چند فاصله یا چند تب پشت سر هم باشند، همه آن‌ها یک توکن خواهند بود.
- جالب اینکه، Galactica تنها مدلی است که دو تب پشت‌سرهم (`t\t`) را نیز به صورت یک توکن واحد رمزگذاری می‌کند.
- مشابه StarCoder2، کد را به طور بهینه مدیریت می‌کند.
- توالی‌های فاصله و تب را به صورت یک توکن واحد نمایش می‌دهد.

🤖 مدل Phi-3 و LLaMA 2

مدل **Phi-3** که در این کتاب بررسی شده، از روش توکن‌سازی مدل **LLaMA 2** استفاده می‌کند، اما در عین حال، توکن‌های ویژه‌ای را نیز به آن اضافه کرده است تا بهتر با کاربردهای گفت‌وگومحور سازگار باشد.

♦ جزئیات توکن‌سازی Phi-3

- روش رمزنگاری: Byte Pair Encoding (BPE)
- تعداد توکن‌ها (واژگان): ۳۲,۰۰۰

♦ توکن‌های ویژه در Phi-3

- نشان‌دهنده پایان متن : `<|endoftext|>`
- توکن‌های گفت‌وگویی (**Chat tokens**):
با محبوب شدن مدل‌های زبانی گفت‌وگومحور در سال ۲۰۲۳، توکن‌سازها برای پشتیبانی بهتر از

گفت‌وگوها، توکن‌هایی را اضافه کردند که نقش **گوینده‌ها و نوبت‌های مکالمه** را مشخص می‌کنند. در Phi-3 نیز چنین توکن‌هایی وجود دارد:

- `<|user|>` : مشخص‌کننده پیام کاربر
- `<|assistant|>` : مشخص‌کننده پاسخ مدل
- `<|system|>` : تنظیمات و دستورالعمل‌های سیستم (مانند نقش‌آفرینی، محدودیت‌ها و راهنماها)

جمع‌بندی

مدل Phi-3 از زیرساخت توکن‌سازی مدل LLaMA 2 بهره می‌برد اما آن را با افزودن توکن‌هایی مخصوص گفت‌وگو گسترش داده است. این تطبیق باعث می‌شود Phi-3 برای سناریوهای گفت‌وگومحور (مثل چت‌بات‌ها) بسیار کارآمد و قابل تنظیم باشد.

مقایسه روشهای توکنایزر

در ادامه، مروری خلاصه و مقایسه‌ای از مدل‌هایی که بررسی کردیم به صورت جدولی آورده شده تا شباهت‌ها و تفاوت‌های آن‌ها در کنار هم به خوبی دیده شوند:

BERT base model (uncased) [CLS] english and capital ##ization [UNK] [UNK] show _ token ##s false none eli ##f == > = else : two tab ##s : " " three tab ##s : " " 12 . 0 * 50 = 600 [SEP]

BERT base model (cased) [CLS] English and CA ##PI ##TA ##L ##I ##Z ##AT ##ION [UNK] [UNK] show _ token ##s F ##als ##e None el ##if == > = else : two ta ##bs : " " Three ta ##bs : " " 12 . 0 * 50 = 600 [SEP]

GPT-2 English and CAP ITAL IZ ATION
 show _ tokens False None el if == > = else : two tabs : " " Three tabs : " " 12 . 0 * 50 = 600

FLAN-T5 English and CA PI TAL IZ ATION <unk> <unk> show _ token s Fal s e None e l if == > = else : two tab s : " " Three tab s : " " 12 . 0 * 50 = 600 </s>

GPT-4 English and CAPITAL IZATION
 show _ tokens False None elif == > = else : two tabs : " " Three tabs : " " 12 . 0 * 50 = 600

StarCoder English and CAPITAL IZATION
 show _ tokens False None elif == > = else : two tabs : " " Three tabs : " " 12 . 0 * 50 = 600

Galactica English and CAP ITAL IZATION
 show _ tokens False None elif == > = else : two t abs : " " Three t abs : " " 12 . 0 * 50 = 600

Phi-3 and Llama 2 <s>
 English and C AP IT AL IZ ATION
 show _ to kens False None elif == > = else : two tabs : " " Three tabs : " " 12 . 0 * 50 = 600

ویژگی‌های توکن‌سازها (Tokenizers) 🌱

توکن‌سازها (Tokenizers) نقش حیاتی در عملکرد مدل‌های زبانی ایفا می‌کنند. آن‌ها تعیین می‌کنند که متن ورودی به چه واحدهایی شکسته شود — یعنی در واقع تعیین می‌کنند که دقیقاً چه چیزی وارد مدل می‌شود. در طول بررسی نمونه‌های مختلف، دیدیم که توکن‌سازها هرکدام رفتار خاص خود را دارند.

توکن‌ساز، فقط ابزاری برای شکستن متن نیست؛ بلکه یکی از تصمیم‌گیری‌های کلیدی در طراحی مدل‌های زبانی است.

روش انتخاب‌شده، تنظیمات آن، و نوع داده‌هایی که برایش هدف‌گذاری شده‌اند، همگی بر دقت و کارآمدی مدل اثر می‌گذارند.

اما سؤال اصلی اینجاست: چه چیزی باعث تفاوت میان توکن‌سازها می‌شود؟

سه عامل اصلی تعیین‌کننده‌ی رفتار آن‌ها عبارت‌اند از:

1 روش توکن‌سازی (Tokenization Method)

هر توکن‌ساز براساس یک الگوریتم خاص کار می‌کند. یکی از رایج‌ترین این الگوریتم‌ها **Byte Pair Encoding (BPE)** است، که در بسیاری از مدل‌های پیشرفته استفاده شده است.

این روش‌ها مشخص می‌کنند که متن چگونه به توکن تبدیل شود، و چه ترکیب‌هایی از حروف یا واژه‌ها در واژگان نهایی گنجانده شوند.

🔍 برای آشنایی با انواع روش‌های توکن‌سازی، می‌توانید به خلاصه مفیدی ([صفحه‌ی Hugging Face](#)) که Hugging Face تهیه کرده مراجعه کنید.

2 تنظیمات توکن‌ساز (Tokenizer Parameters)

پس از انتخاب روش، طراح مدل باید تنظیمات مهمی را مشخص کند. مهم‌ترین آن‌ها:

اندازه واژگان (Vocabulary Size)

تعداد توکن‌هایی که در واژگان مدل قرار می‌گیرند.

- رایج‌ترین اندازه‌ها: ۳۰,۰۰۰ یا ۵۰,۰۰۰
- در مدل‌های جدیدتر، اندازه‌هایی تا ۱۰۰,۰۰۰ نیز دیده می‌شود.

توکن‌های ویژه (Special Tokens)

مدل‌ها برای انجام وظایف خاص نیاز به توکن‌هایی دارند که مفاهیمی مانند آغاز و پایان متن، پرکننده‌ها، یا واژه‌های ناشناخته را مشخص کنند. از جمله:

- `<s>` : آغاز متن
- `</s>` : پایان متن
- `<pad>` : پرکننده (برای هم‌تراز کردن توالی‌ها)
- `<unk>` : واژه‌ای که در واژگان مدل نیست
- `[CLS]` : برای طبقه‌بندی
- `[MASK]` : BERT برای ماسک‌کردن واژه‌ها در مدل‌هایی مثل

همچنین، بسته به کاربرد، می‌توان توکن‌های خاص دامنه‌ای تعریف کرد.
مثلاً:

- مدل **Galactica** از توکن‌هایی مثل [START_REF] برای ارجاعات علمی و <work> برای نمایش استدلال مرحله‌ای استفاده می‌کند.

✨ حساسیت به حروف بزرگ (Capitalization)

در زبان‌هایی مثل انگلیسی، **بزرگ یا کوچک بودن حروف** می‌تواند اطلاعات مهمی منتقل کند (مثلاً نام‌ها، اختصارات، یا تأکید). اما این موضوع چالشی در طراحی توکن‌ساز ایجاد می‌کند:

- آیا بهتر است همه واژه‌ها به حروف کوچک تبدیل شوند؟
 - یا اینکه نسخه‌های بزرگ و کوچک هر واژه، به صورت توکن‌های جداگانه در واژگان مدل حفظ شوند؟
- ⚖️ این تصمیم به تعادل میان دقت معنایی و صرفه‌جویی در فضای واژگان بستگی دارد.

3 دامنه‌ی داده (Domain of Data)

حتی اگر دو توکنایزر از یک روش و پارامترهای مشابه استفاده کنند، رفتار توکن‌ساز بسته به **داده‌هایی که برای آموزش آن استفاده شده** کاملاً متفاوت خواهد بود. دلیل این امر این است که روش‌های توکن‌سازی معمولاً واژگان را به گونه‌ای بهینه می‌کنند که بتوانند **داده‌های خاصی** را به بهترین شکل نمایش دهند. همان‌طور که دیدیم، این موضوع روی داده‌هایی مثل کد برنامه‌نویسی یا متون چندزبانه تأثیر زیادی دارد.

مثال از تفاوت توکن‌سازی در داده‌های کدنویسی

فرض کنید یک توکن‌ساز عادی که برای متن نوشته شده، متن زیر را توکنیزه می‌کند:

```
def add_numbers(a, b):  
    ..."""Add the two numbers `a` and `b`."""  
    ...return a + b
```

در اینجا، فاصله‌های تو رفتگی (indentation) ممکن است به صورت توکن‌های جداگانه و پراکنده تبدیل شوند که می‌تواند در مدل‌سازی کد، کارایی بهینه‌ای نداشته باشد.

اما یک توکن‌ساز اختصاصی برای کد، این فضاها را به صورت بهینه‌تر و یکپارچه‌تر توکنیزه می‌کند:

```
def add_numbers(a, b):  
    ...."""Add the two numbers `a` and `b`."""  
    ....return a + b
```

این انتخاب بهتر در توکن‌سازی باعث می‌شود مدل راحت‌تر ساختار کد را درک کند و در نتیجه احتمال بهبود عملکرد مدل افزایش یابد.

برای یادگیری عمیق‌تر درباره‌ی آموزش توکنایزرها، می‌توانید به بخش Tokenizers در دوره‌ی Hugging Face و کتاب Natural Language Processing with Transformers, Revised Edition مراجعه کنید.

توکن امبدینگ‌ها (Token Embeddings)

حالا که مفهوم توکنیزه کردن را فهمیدیم، یک بخش از چالش تبدیل زبان به داده‌ای قابل فهم برای مدل‌های زبانی حل شده است. زبان، در این دیدگاه، رشته‌ای از توکن‌هاست. اگر مدل به اندازه کافی قوی باشد و روی مجموعه بزرگی از این توکن‌ها آموزش ببیند، به تدریج الگوهای پیچیده موجود در داده‌های آموزشی را یاد می‌گیرد:

- اگر داده‌های آموزش شامل حجم زیادی متن انگلیسی باشد، مدل قادر می‌شود زبان انگلیسی را بفهمد و تولید کند.
- اگر داده‌های آموزش شامل اطلاعات واقعی و مستند مانند ویکی‌پدیا باشد، مدل توانایی تولید اطلاعات واقعی را پیدا می‌کند (البته با محدودیت‌هایی که در ادامه اشاره می‌کنیم).

اما قدم بعدی در این معما، پیدا کردن بهترین نمایش عددی (numerical representation) برای هر توکن است تا مدل بتواند با آن اعداد محاسباتش را انجام دهد و الگوهای متن را بهتر یاد بگیرد.

این الگوها همان چیزی هستند که باعث می‌شوند مدل به صورت منسجم در یک زبان صحبت کند، یا بتواند کد برنامه‌نویسی بنویسد، یا مهارت‌های دیگری که از مدل‌های زبانی انتظار داریم را کسب کند.

همان‌طور که در فصل اول دیدیم، این نمایش‌های عددی همان امبدینگ‌ها (Embeddings) هستند. امبدینگ‌ها فضای عددی هستند که معانی و روابط بین کلمات و توکن‌ها را در خود ذخیره می‌کنند.

نکته مهم

با پیشرفت مدل‌ها و افزایش توانایی آن‌ها در تولید متن‌های منسجم و حتی تولید اطلاعات واقعی، برخی کاربران شروع به اعتماد زیاد به این توانایی‌ها کردند. مثلاً در اوایل ۲۰۲۳، برخی مدل‌های زبانی به عنوان «قاتل گوگل» (Google Killers) مطرح شدند!

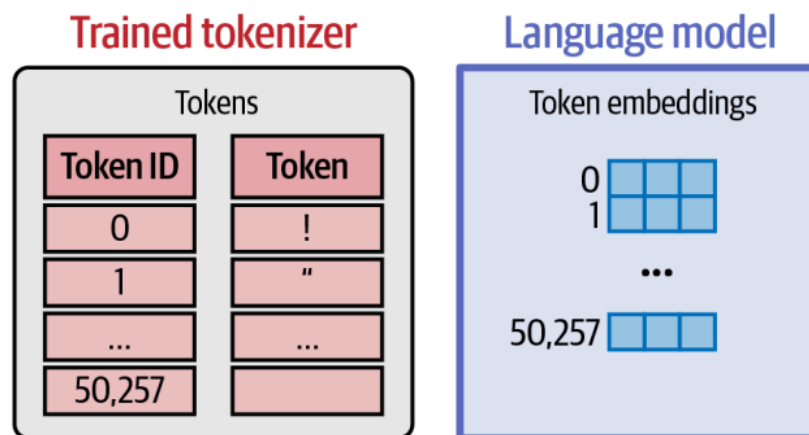
اما خیلی زود مشخص شد که مدل‌های زبانی به تنهایی جایگزین موتورهای جستجو نیستند و ممکن است اطلاعات نادرست یا گمراه‌کننده تولید کنند.

این مسئله باعث شد روش‌های جدیدی مانند تولید مبتنی بر بازیابی اطلاعات (Retrieval-Augmented Generation یا RAG) شکل بگیرد که ترکیبی از جستجو و مدل‌های زبانی است. درباره RAG در فصلهای آینده توضیح بیشتری خواهیم داد.

مدل زبانی بردارهای امبدینگ شده (Embedding) را برای واژگان توکنایزر خود نگه می‌دارد

پس از این‌که یک توکنایزر مقداردهی اولیه و آموزش داده شد، از آن در مرحله آموزش مدل زبان مرتبط با آن استفاده می‌شود. به همین دلیل، یک مدل زبان pretrained همیشه به توکن‌ساز خودش وابسته است و نمی‌تواند بدون آموزش مجدد، با توکن‌ساز دیگری کار کند.

هر مدل زبان برای هر توکن در واژگان توکن‌ساز، یک بردار امبدینگ دارد — همان‌طور که در شکل زیر می‌بینیم. وقتی مدل از پیش آموزش داده شده ای (pretrained) را دانلود می‌کنیم، بخشی از آن شامل ماتریسی است که این بردارهای امبدینگ را در خود جای داده است.



در ابتدا، این بردارها به صورت تصادفی مقداردهی می‌شوند، مانند سایر وزن‌های مدل، اما طی فرایند آموزش، مقادیر این بردارها به گونه‌ای تنظیم می‌شوند که مدل بتواند به شکل مؤثری وظایف خود را انجام دهد و زبان را به خوبی درک کند.

ساخت امبدینگ‌های زمینه‌ای (Contextualized Word Embeddings) با مدل‌های زبانی

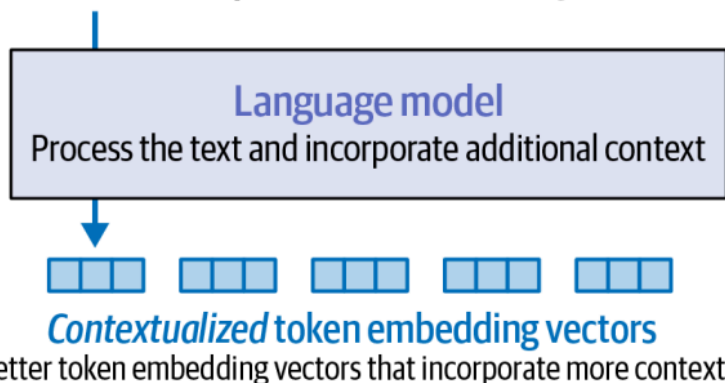
حالا که با امبدینگ‌های توکن به عنوان ورودی مدل‌های زبانی آشنا شدیم، بیایید ببینیم چگونه این مدل‌ها می‌توانند امبدینگ‌های دقیق‌تر و هوشمندانه‌تری بسازند. این روش یکی از اصلی‌ترین راه‌ها برای نمایش بهتر متن است.

به جای اینکه هر کلمه یا توکن یک بردار ثابت داشته باشد، مدل‌های زبانی امبدینگ‌های زمینه‌ای ایجاد می‌کنند؛ یعنی برای هر کلمه، بسته به موقعیت و محیط اطرافش، یک بردار متفاوت تولید می‌کنند (contextualized token embeddings) (همان‌طور که در شکل زیر می‌بینید).

این بردارهای پویا و هوشمند، به سیستم‌های مختلف امکان می‌دهند تا در کارهای متنوعی مثل شناسایی اسامی خاص (Named-Entity Recognition) یا خلاصه‌سازی استخراجی متن (که بخش‌های مهم یک متن طولانی را جدا می‌کند) عملکرد بهتری داشته باشند.

نکته جالب این است که همین نوع بردارهای زمینه‌ای، نیروی محرکه پشت سیستم‌های هوش مصنوعی تولید تصویر مثل **DALL-E**، **Midjourney** و **Stable Diffusion** هم هستند.

Have the bards who preceded me left any theme unsung?



بیایید ببینیم چگونه می‌توانیم امبدینگ‌های زمینه‌ای برای کلمات تولید کنیم

```
from transformers import AutoModel, AutoTokenizer

# Load a tokenizer
tokenizer = AutoTokenizer.from_pretrained("microsoft/deberta-base")

# Load a language model
model = AutoModel.from_pretrained("microsoft/deberta-v3-xsmall")

# Tokenize the sentence
tokens = tokenizer('Hello world', return_tensors='pt')

# Process the tokens
output = model(tokens)[0]
```

مدلی که ما در اینجا استفاده می‌کنیم DeBERTa v3 نام دارد که در زمان نوشتن این متن، یکی از بهترین مدل‌های زبانی برای بردارهای تعبیه (embedding) توکن است و در عین حال کوچک و بسیار کارآمد است. این مدل در مقاله "DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training gradient-disentangled embedding sharing" توضیح داده شده است. ([آدرس مقاله](#))

این کد ابتدا توکن‌ساز و مدل pretrained را دانلود می‌کند، سپس آن‌ها را برای پردازش رشته‌ی «Hello world» استفاده می‌کند. خروجی مدل در متغیری به نام output ذخیره می‌شود. بیایید ابتدا ابعاد این

خروجی را بررسی کنیم (که انتظار داریم آرایه‌ای چندبعدی باشد):

```
output.shape
```

خروجی این دستور به شکل زیر است:

```
torch.Size([1, 4, 384])
```

اگر بعد اول را نادیده بگیریم، این یعنی مدل چهار توکن دریافت کرده که هر کدام به یک بردار ۳۸۴ بعدی تبدیل شده‌اند. بعد اول نشان‌دهنده اندازه‌ی دسته (batch size) است که در مواردی مانند آموزش استفاده می‌شود، وقتی می‌خواهیم چند جمله را همزمان به مدل بدهیم تا پردازش سریع‌تر انجام شود.

اما این چهار بردار چه هستند؟ آیا توکن‌ساز دو کلمه «Hello» و «world» را به چهار توکن شکسته است یا دلیل دیگری دارد؟ با استفاده از دانش خود درباره توکن‌سازها، می‌توانیم آن‌ها را بررسی کنیم:

```
for token in tokens['input_ids'][0]:  
    print(tokenizer.decode(token))
```

خروجی این کد چنین است:

```
[CLS]  
Hello  
world  
[SEP]
```

این توکن‌ساز و مدل خاص، توکن‌های [CLS] و [SEP] را به ترتیب در ابتدا و انتهای متن اضافه می‌کنند.

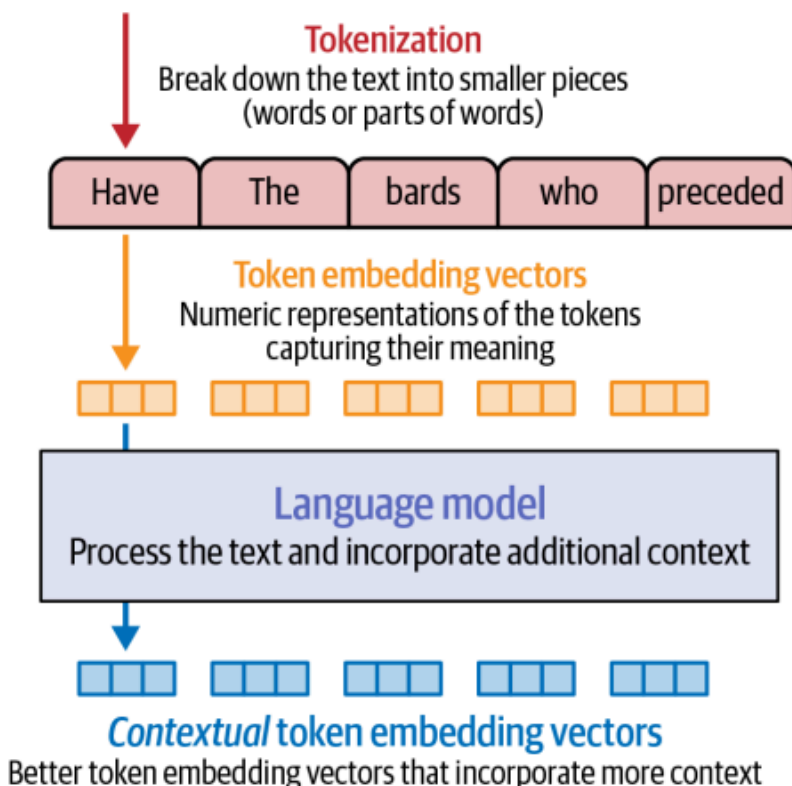
حالا مدل زبان متن را پردازش کرده است و خروجی آن چیزی شبیه این است:

```
tensor([[  
  [-3.3060, -0.0507, -0.1098, ..., -0.1704, -0.1618, 0.6932],  
  [ 0.8918, 0.0740, -0.1583, ..., 0.1869, 1.4760, 0.0751],  
  [ 0.0871, 0.6364, -0.3050, ..., 0.4729, -0.1829, 1.0157],  
  [-3.1624, -0.1436, -0.0941, ..., -0.0290, -0.1265, 0.7954]  
]], grad_fn=<NativeLayerNormBackward0>)
```

این همان خروجی خام مدل زبان است. بسیاری از کاربردهای مدل‌های بزرگ زبانی بر پایه همین خروجی‌ها ساخته می‌شوند.

ما خلاصه‌ای از فرایند توکنیزه کردن ورودی و خروجی‌های مدل زبان را در شکل زیر می‌بینیم. به طور فنی، تبدیل شناسه‌های توکن به امبدینگ‌های خام اولین مرحله‌ای است که داخل مدل زبان اتفاق می‌افتد.

Have the bards who preceded me left any theme unsung?

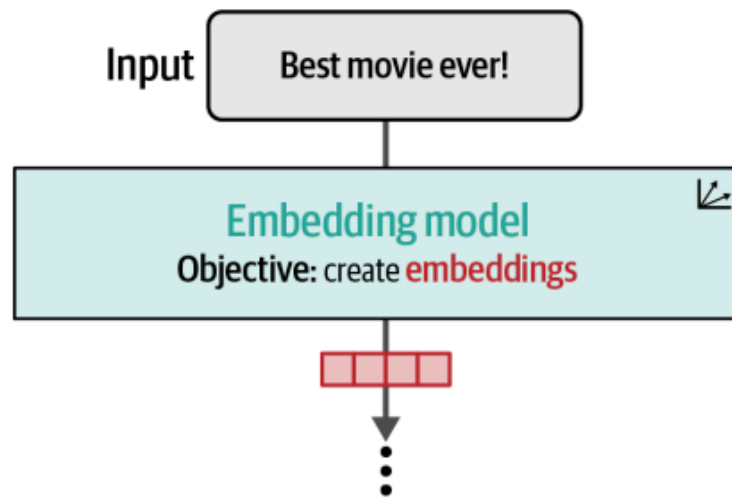


این تصویر برای فصل بعدی که به بررسی چگونگی کارکرد مدل‌های مبتنی بر ترنسفورمر می‌پردازیم، بسیار مهم و کاربردی خواهد بود.

امبدینگ‌های متنی برای جملات و اسناد کامل

اگرچه امبدینگ‌های توکن اساس کار مدل‌های زبانی بزرگ هستند، اما در بسیاری از کاربردهای این مدل‌ها نیاز دارند که روی جملات، پاراگراف‌ها یا حتی اسناد کامل کار کنند. به همین دلیل، مدل‌هایی ساخته شده‌اند که **امبدینگ‌های متنی** تولید می‌کنند؛ یعنی یک بردار واحد که نمایانگر معنای یک قطعه متن بلندتر است.

می‌توان این‌گونه تصور کرد که مدل، متن ورودی را گرفته و در نهایت یک بردار عددی می‌سازد که معنی و مفهوم و ویژگی‌های آن متن را به شکل فشرده و کاربردی در خود جای داده است. شکل زیر این فرایند را به خوبی نشان می‌دهد.



راه‌های مختلفی برای ساخت این بردارها وجود دارد؛ یکی از ساده‌ترین آن‌ها میانگین گرفتن از بردارهای تمام توکن‌های متن است. اما مدل‌های پیشرفته‌تر، به طور ویژه برای این کار آموزش دیده‌اند و نتایج بهتری ارائه می‌دهند.

یکی از ابزارهای محبوب برای تولید امبدینگ‌های متنی، بسته‌ی **sentence-transformers** است که مدل‌های آماده و قدرتمندی را ارائه می‌دهد. مانند مدل‌های ترنسفورمر که در فصل قبل آموختیم، این بسته هم به آسانی مدل‌های pretrained را بارگذاری می‌کند.

برای نمونه، با مدل **all-mpnet-base-v2**، متن زیر را به یک بردار امبدینگ تبدیل می‌کنیم:

```
from sentence_transformers import SentenceTransformer

# بارگذاری مدل
model = SentenceTransformer("sentence-transformers/all-mpnet-base-v2")

# تبدیل متن به بردار امبدینگ
vector = model.encode("Best movie ever!")
```

ابعاد بردار امبدینگ به مدل استفاده شده بستگی دارد؛ برای این مدل:

```
vector.shape
```

خروجی:

```
(768,)
```

یعنی جمله‌ی ما در قالب یک بردار ۷۶۸ بعدی نمایان شده است که معنی و ویژگی‌های آن را در خود دارد. یعنی این جمله اکنون در این یک بردار با ابعاد 768 مقادیر عددی رمزگذاری شده است.

در بخش بعدی، وقتی سراغ کاربردهای عملی می‌رویم، خواهید دید که این بردارهای امبدینگ چقدر در زمینه‌هایی مثل دسته‌بندی، جستجوی معنایی و روش‌های ترکیبی مثل RAG اهمیت دارند.

امبدینگ‌های واژه فراتر از مدل‌های زبانی بزرگ

امبدینگ‌ها فقط در حوزه‌ی متن و تولید زبان کاربرد ندارند. اختصاص دادن نمایش‌های عددی معنادار به اشیاء یا داده‌ها، در بسیاری از زمینه‌ها مثل موتورهای پیشنهادگر و رباتیک هم بسیار مفید است. در این بخش، به استفاده از امبدینگ‌های pretrained مانند **word2vec** می‌پردازیم و کمی درباره نحوه آموزش آن‌ها صحبت می‌کنیم. آشنایی با روش آموزش word2vec، زمینه‌ی خوبی برای درک **روش‌های آموزشی مبتنی بر تضاد (contrastive training)** است که در فصل دهم بررسی می‌کنیم. سپس می‌بینیم چگونه این امبدینگ‌ها در سیستم‌های پیشنهاددهی کاربرد دارند.

استفاده از امبدینگ‌های واژه از پیش آموزش داده شده (pretrained)

بیاید ببینیم چطور می‌توانیم امبدینگ‌های pretrained مانند word2vec یا GloVe را با استفاده از کتابخانه‌ی **Gensim** دانلود کنیم:

```
import gensim.downloader as api

# آموزش‌دیده روی ویکی‌پدیا، اندازه بردار: ۶۶، GloVe، (۵۰ مگابایت)
# نیز موجود است "word2vec-google-news-300" گزینه‌های دیگر مثل
# https://github.com/RaRe-Technologies/gensim-data برای مشاهده گزینه‌های بیشتر به
# مراجعه کنید

model = api.load("glove-wiki-gigaword-50")
```

در اینجا، ما لیستی از امبدینگ‌های تعداد زیادی کلمه را که روی ویکی‌پدیا آموزش دیده‌اند، دانلود کرده‌ایم. حالا می‌توانیم فضای امبدینگ‌ها را با دیدن نزدیک‌ترین همسایه‌ها به یک کلمه خاص مثل "king" بررسی کنیم:

```
model.most_similar([model['king']], topn=11)
```

خروجی چیزی شبیه به این خواهد بود:

```
[('king', 1.0000),
 ('prince', 0.8236),
 ('queen', 0.7839),
 ('ii', 0.7746),
 ('emperor', 0.7736),
 ('son', 0.7667),
 ('uncle', 0.7627),
```

```
('kingdom', 0.7542),  
( 'throne', 0.7540),  
( 'brother', 0.7492),  
( 'ruler', 0.7434)]
```

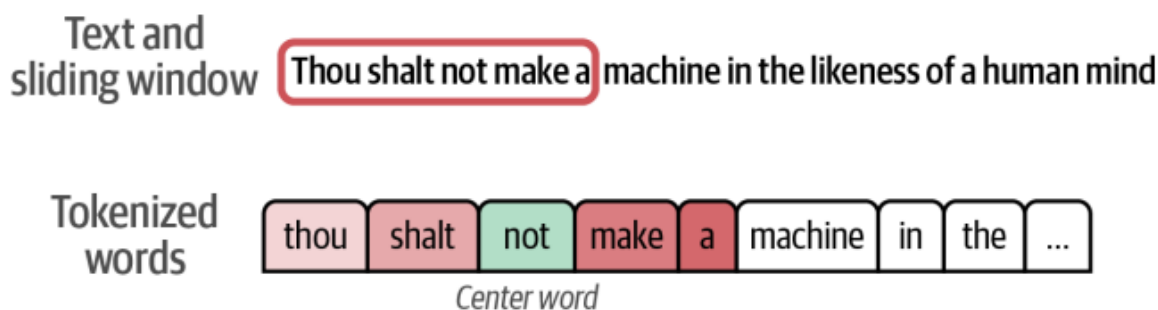
این نتایج نشان می‌دهد کلمات معنایی مشابه یا مرتبط در فضای برداری به هم نزدیک هستند؛ مثلاً کلماتی مثل "prince" (شاهزاده)، "queen" (ملکه) و "throne" (تخت) در مجاورت "king" قرار دارند.

الگوریتم Word2vec و آموزش مبتنی بر تضاد (Contrastive Training)

الگوریتم word2vec که در [مقاله](#) شرح داده شده است و در بخش‌های ضمیمه نیز به آن پرداختیم. اینجا خلاصه‌ای از اصول آن را مرور می‌کنیم چون در بخش‌های بعدی، به خصوص درباره‌ی سیستم‌های پیشنهاددهی، از همین مفاهیم استفاده خواهیم کرد. مثل مدل‌های زبانی بزرگ (LLM)، Word2vec هم با استفاده از نمونه‌هایی که از متن تولید می‌شود، آموزش می‌بیند. فرض کنید متنی از رمان‌های «تل‌ماسه» نوشته فرانک هربرت داریم:

"Thou shalt not make a machine in the likeness of a human mind."

الگوریتم با استفاده از یک پنجره‌ی لغزنده (sliding window) نمونه‌های آموزشی را می‌سازد. مثلاً اگر اندازه پنجره را ۲ قرار دهیم، یعنی دو کلمه سمت چپ و دو کلمه سمت راست هر کلمه مرکزی را به عنوان ورودی مدل، در نظر می‌گیریم.



Thou shalt not make a machine in the likeness of a human mind

Sliding window across running text

thou	shalt	not	make	a	machine	in	the	...
------	-------	-----	------	---	---------	----	-----	-----

Dataset

input 1	input 2	output
thou	shalt	not

Thou shalt not make a machine in the likeness of a human mind

Sliding window across running text

thou	shalt	not	make	a	machine	in	the	...
thou	shalt	not	make	a	machine	in	the	

Dataset

input 1	input 2	output
thou	shalt	not
shalt	not	make

هدف مدل این است که یاد بگیرد آیا دو کلمه معمولاً در کنار هم (در یک زمینه) ظاهر می‌شوند یا خیر. به این ترتیب، مدل دو کلمه را می‌گیرد و تلاش می‌کند بگوید آیا این جفت کلمات همسایه هستند یا نه.

در این روش، امبدینگ‌ها با استفاده از یک وظیفه‌ی دسته‌بندی (classification) ساخته می‌شوند به این صورت که یک شبکه عصبی آموزش می‌بیند که پیش‌بینی کند آیا دو کلمه معمولاً در یک زمینه‌ی مشترک (context) ظاهر می‌شوند یا نه. می‌توانیم این شبکه را به صورت یک تابع ببینیم که دو کلمه به عنوان ورودی می‌گیرد و اگر آن دو کلمه در کنار هم ظاهر شوند خروجی ۱ و در غیر این صورت ۰ می‌دهد.

در هر نمونه‌ی آموزشی، کلمه‌ی مرکزی یک ورودی است و هر کدام از کلمات همسایه به عنوان ورودی دوم در نمونه‌های جداگانه استفاده می‌شود. هدف این است که مدل یاد بگیرد رابطه‌ی همسایگی بین دو کلمه را تشخیص دهد و اگر دو کلمه واقعاً همسایه باشند، خروجی ۱ بدهد. (به شکل زیر توجه نمایید)

Word1	Word2	Target
Not	thou	1
Not	shalt	1
Not	make	1
Not	a	1

Training examples

اما اگر همه‌ی نمونه‌ها فقط شامل برچسب مثبت (یا همان مقدار ۱ در تصویر بالا) داشته باشند، مدل به راحتی می‌تواند همیشه عدد ۱ را پیش‌بینی کند. برای جلوگیری از این مشکل، باید نمونه‌های منفی (negative)

examples) به داده های آموزش اضافه کنیم؛ یعنی جفت کلماتی که معمولاً کنار هم نیستند (شکل زیر):

Word 1	Word 2	Target
not	thou	1
not	shalt	1
not	make	1
not	a	1
thou	apothecary	0
not	sublime	0
make	def	0
a	playback	0

Positive examples

Negative examples

در انتخاب نمونه های منفی لازم نیست خیلی دقیق باشیم. یکی از ایده های مهمی که اینجا مطرح است، برآورد تضاد با نویز (noise-contrastive estimation) است. به همین دلیل، گرفتن کلمات تصادفی به عنوان نمونه های منفی و مشخص کردن آن ها به عنوان داده هایی که با یکدیگر در یک پنجره لغزنده «همسایه نیستند» به مدل کمک می کند عملکرد بهتری داشته باشد.

به این ترتیب، دو مفهوم کلیدی Word2vec را در شکل زیر می بینیم :

- **Skip-gram:** انتخاب کلمات همسایه در پنجره لغزنده
- **Negative sampling:** نمونه برداری منفی برای افزودن جفت های منفی

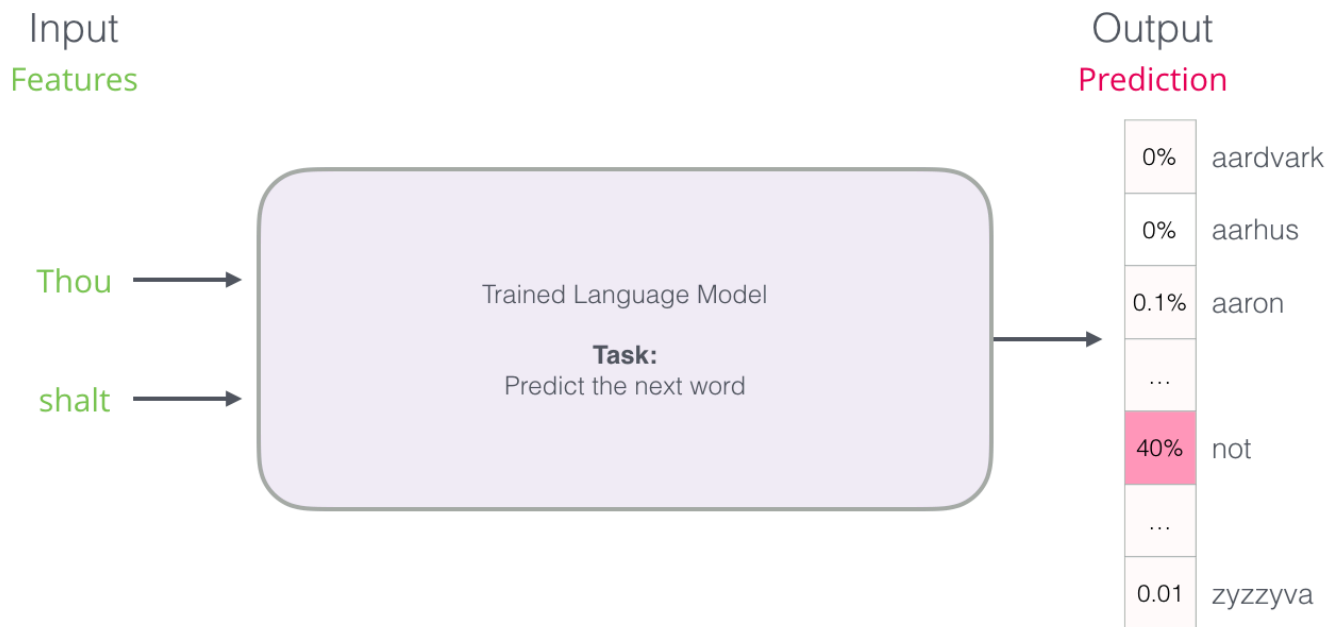
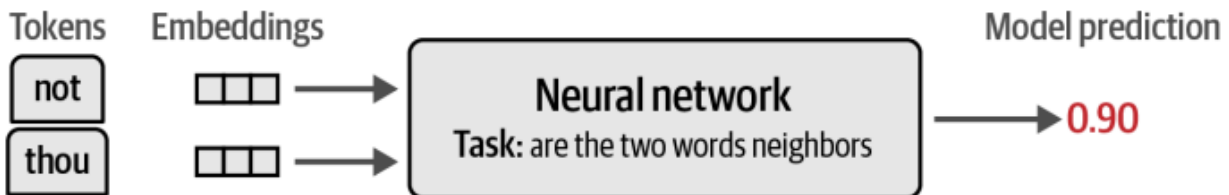
Skip-gram					Negative sampling		
shalt	not	make	a	machine	Input word	Output word	Target
input		output			make	shalt	1
make		shalt			make	aaron	0
make		not			make	taco	0
make		a					
make		machine					

با این روش می توان میلیون ها نمونه ی آموزشی از متن های پیوسته ساخت. قبل از آموزش شبکه عصبی روی این داده ها، باید تصمیماتی درباره توکنیزه کردن متن بگیریم؛ مانند نحوه برخورد با حروف بزرگ و علائم نگارشی و تعیین اندازه دیکشنری (Vocabulary).

سپس برای هر توکن، یک بردار امبدینگ به صورت تصادفی مقداردهی اولیه می کنیم:

Token	Token embedding
thou	
shalt	
make	
a	
not	
apothecary	
sublime	
def	
playback	

مدل روی هر نمونه آموزش می‌بیند که دو بردار امبدینگ ورودی را گرفته و پیش‌بینی کند آیا این دو کلمه همسایه هستند یا نه



هر بار که پیش‌بینی مدل درست یا غلط باشد، وزن‌ها (بردارهای امبدینگ) به‌روزرسانی می‌شوند تا در دفعات بعدی پیش‌بینی دقیق‌تر باشد. در نهایت، بردارهای امبدینگ به شکلی آموزش دیده می‌شوند که معنی و زمینه کلمات را بهتر بازتاب دهند.

این ایده‌ی آموزش مدلی که دو بردار را می‌گیرد و رابطه‌ی بین آن‌ها را پیش‌بینی می‌کند، یکی از روش‌های قدرتمند یادگیری ماشین است و بارها در مدل‌های زبانی و کاربردهای دیگر اثربخش بوده. در فصل‌های بعدی به این روش‌ها و کاربردهایشان در بهینه‌سازی مدل‌های زبانی خواهیم پرداخت.

همچنین این تکنیک، پایه‌ی مدل‌های چندرسانه‌ای است که متن و تصویر را به هم وصل می‌کنند؛ مثلاً مدلی که تشخیص می‌دهد آیا یک کپشن متنی، تصویر مربوطه را به درستی توصیف می‌کند یا خیر. درباره این موضوع در فصل‌های بعد بیشتر صحبت خواهیم کرد.

امبدینگ‌ها در سیستم‌های پیشنهاددهی

همان‌طور که اشاره کردیم، مفهوم امبدینگ‌ها (بردارهای عددی که نمایش معنایی یک عنصر هستند) کاربردهای گسترده‌ای در زمینه‌های مختلف دارد. یکی از مهم‌ترین کاربردهای صنعتی آن‌ها در سیستم‌های پیشنهاددهی است.

پیشنهاد آهنگ با استفاده از امبدینگ‌ها

در این بخش، از الگوریتم **word2vec** برای ساخت امبدینگ آهنگ‌ها استفاده می‌کنیم — با این فرض که هر آهنگ مثل یک «کلمه» و هر فهرست پخش (**playlist**) مثل یک «جمله» است. با این کار، آهنگ‌هایی که معمولاً با هم در یک لیست پخش ظاهر می‌شوند، به صورت برداری به هم نزدیک‌تر خواهند شد. در نتیجه، سیستم می‌تواند آهنگ‌های مشابه را به خوبی پیشنهاد دهد.

دیتاست مورد استفاده توسط *Shuo Chen* از دانشگاه کرنل گردآوری شده و شامل فهرست‌های پخش رادیویی از صدها ایستگاه در سراسر آمریکاست. شکل زیر نمایی از ساختار این داده‌ها را نشان می‌دهد: هر لیست پخش شامل مجموعه‌ای از آهنگ‌هاست.

Playlist #1:	Song 1	Song 13	Song 2	Song 400	
Playlist #2:	Song 2	Song 81	Song 13	Song 82	Song 77
Playlist #3:	Song 13	Song 2			

نگاهی به نتیجه نهایی (قبل از اجرای کد)

پیش از آنکه وارد جزئیات ساخت این سیستم شویم، بیایید خروجی آن را ببینیم. فرض کنید آهنگی را به سیستم می‌دهیم و از آن می‌خواهیم آهنگ‌های مشابه را پیشنهاد دهد.

برای مثال، آهنگ معروف "**Billie Jean**" از *Michael Jackson* را با شناسه 3822 به سیستم می‌دهیم:

```
print_recommendations(3822)
```

خروجی:

id	Title	Artist
4181	Kiss	Prince & The Revolution
12749	Wanna Be Startin' Somethin'	Michael Jackson

1506	The Way You Make Me Feel	Michael Jackson
3396	Holiday	Madonna
500	Don't Stop 'Til You Get Enough	Michael Jackson

پاسخ منطقی به نظر می‌رسد: آهنگ‌های دیگر از مایکل جکسون، و همچنین آهنگ‌هایی از هنرمندان هم‌سبک مانند *Prince* و *Madonna*.

حالا برویم به سراغ سبک رپ. این بار آهنگ **"California Love"** از *2Pac* با شناسه 842 را امتحان می‌کنیم:

```
print_recommendations(842)
```

خروجی:

id	Title	Artist
413	If I Ruled the World (Imagine That)	Nas (feat. Lauryn Hill)
196	I'll Be Missing You	Puff Daddy & The Family
330	Hate It or Love It	The Game (feat. 50 Cent)
211	Hypnotize	The Notorious B.I.G.
5788	Drop It Like It's Hot	Snoop Dogg (feat. Pharrell)

باز هم پیشنهادهایی مناسب و منطقی دریافت می‌کنیم.

در ادامه، قدم به قدم خواهیم دید که این سیستم چگونه ساخته شده، چطور داده‌ها را به شکل مناسب تبدیل می‌کنیم، و چگونه از word2vec برای یادگیری شباهت بین آهنگ‌ها بهره می‌بریم.

آموزش یک مدل امبدینگ برای آهنگ‌ها

برای آموزش مدلی که بتواند آهنگ‌های مشابه را شناسایی کند، ابتدا دیتاست لیست‌های پخش (playlist) و اطلاعات آهنگ‌ها (عنوان و نام هنرمند) را بارگذاری می‌کنیم:

```
import pandas as pd
from urllib import request

# دریافت فایل لیست پخش آهنگ‌ها
data = request.urlopen('https://storage.googleapis.com/maps-premium/dataset/yes_complete/train.txt')
```

```

پردازش فایل لیست‌ها: دو خط ابتدایی شامل متادیتا هستند و حذف می‌شوند
lines = data.read().decode("utf-8").split('\n')[2:]

حذف لیست‌هایی که فقط شامل یک آهنگ هستند
playlists = [s.rstrip().split() for s in lines if len(s.split()) > 1]

بارگذاری اطلاعات آهنگ‌ها (شناسه، عنوان، هنرمند)
songs_file = request.urlopen('https://storage.googleapis.com/maps-
premium/dataset/yes_complete/song_hash.txt')
songs_file = songs_file.read().decode("utf-8").split('\n')
songs = [s.rstrip().split('\t') for s in songs_file]
songs_df = pd.DataFrame(data=songs, columns = ['id', 'title', 'artist'])
songs_df = songs_df.set_index('id')

```

اکنون می‌توانیم محتوای لیست‌ها را مشاهده کنیم:

```

print('Playlist #1:\n', playlists[0], '\n')
print('Playlist #2:\n', playlists[1])

```

آموزش مدل Word2Vec برای امبدینگ آهنگ‌ها

برای آموزش مدل، از الگوریتم Word2Vec استفاده می‌کنیم. در اینجا، هر آهنگ مانند یک «کلمه» و هر لیست پخش مانند یک «جمله» در نظر گرفته می‌شود:

```

from gensim.models import Word2Vec

# با تنظیمات مناسب Word2Vec آموزش مدل
model = Word2Vec(
    playlists,
    vector_size=32,
    window=20,
    negative=50,
    min_count=1,
    workers=4
)

```

پس از چند دقیقه، مدل آموزش می‌بیند و برای هر آهنگ یک بردار امبدینگ می‌سازد.

اکنون می‌توانیم آهنگ‌های مشابه با آهنگ خاصی را پیدا کنیم. مثلاً برای آهنگ با شناسه 2172 :

```
song_id = 2172
model.wv.most_similar(positive=str(song_id))
```

خروجی، فهرستی از آهنگ‌هایی است که بیشترین شباهت برداری را با آهنگ موردنظر دارند:

```
[('2976', 0.9977),
 ('3167', 0.9977),
 ('3094', 0.9976),
 ('2640', 0.9966),
 ('2849', 0.9963)]
```

این آهنگ مربوط به Metallica است:

```
print(songs_df.iloc[2172])
# title: Fade To Black
# artist: Metallica
```

همان‌طور که انتظار می‌رود، آهنگ‌های پیشنهادی همگی در سبک هارد راک و هوی‌متال هستند.

تابعی برای نمایش آهنگ‌های پیشنهادی

```
import numpy as np

def print_recommendations(song_id):
    similar_songs = np.array(
        model.wv.most_similar(positive=str(song_id), topn=5)
    )[:, 0]
    return songs_df.loc[similar_songs]

# نمایش آهنگ‌های مشابه با آهنگ Metallica
print_recommendations(2172)
```

خروجی ممکن است چیزی شبیه این باشد:

ID	عنوان	هنرمند
11473	Little Guitars	Van Halen
3167	Unchained	Van Halen
5586	The Last in Line	Dio

ID	عنوان	هنرمند
5634	Mr. Brownstone	Guns N' Roses
3094	Breaking the Law	Judas Priest

این نشان می‌دهد که مدل به‌درستی توانسته است آهنگ‌های هم‌سبک را شناسایی کند.

جمع‌بندی

در این فصل، به بررسی مفاهیم کلیدی مربوط به توکن‌ها، توکنایزرها و روش‌های کاربردی استفاده از **امبدینگ‌های توکن** در مدل‌های زبانی بزرگ (LLM) پرداختیم. این مطالب نه‌تنها ما را برای درک بهتر عملکرد مدل‌های زبانی در فصل بعد آماده می‌کنند، بلکه دریچه‌ای به کاربرد گسترده‌ی امبدینگ‌ها فراتر از مدل‌های زبانی باز می‌کنند.

خلاصه‌ی مفاهیم کلیدی فصل:

♦ **توکنایزرها** نخستین گام در پردازش ورودی به مدل زبانی هستند. آن‌ها متن خام را به توکن‌ها یا شناسه‌هایی قابل فهم برای مدل تبدیل می‌کنند. بسته به نیاز، این تبدیل می‌تواند در سطح واژه، زیرواژه، نویسه یا بایت انجام شود.

♦ **در مرور توکنایزهای واقعی** مانند GPT-4، GPT-2، BERT و مدل‌های دیگر دیدیم که:

- برخی توکنایزرها بهتر اطلاعاتی مثل **حروف بزرگ، خطوط جدید، یا زبان‌های دیگر** را حفظ می‌کنند.
- برخی دیگر صرفاً متفاوت عمل می‌کنند، مثلاً در نحوه‌ی شکستن کلمات خاص.

♦ **سه تصمیم کلیدی در طراحی توکنایزر:**

1. **الگوریتم توکن‌سازی** (مثل BPE، WordPiece، SentencePiece)
2. **پارامترهای توکنایزر** (اندازه واژگان، توکن‌های خاص، نحوه‌ی برخورد با حروف بزرگ و زبان‌ها)
3. **دامنه‌ی داده‌های آموزشی** که توکنایزر بر اساس آن آموزش دیده است.

امبدینگ‌های توکن

مدل‌های زبانی قادرند **امبدینگ‌های متنی باکیفیت و زمینه‌مند** بسازند که از امبدینگ‌های ایستا بسیار قدرتمندترند. این بردارها اساس بسیاری از وظایف NLP هستند، مانند:

- شناسایی موجودیت‌های نامدار (NER)
- خلاصه‌سازی استخراجی
- طبقه‌بندی متون

افزون بر این، مدل‌های زبانی می‌توانند بردارهایی برای جملات یا اسناد کامل تولید کنند، که مبنای بسیاری از کاربردهای مدرن مثل جستجوی معنایی، طبقه‌بندی معنایی و RAG هستند. در بخش دوم کتاب، به این کاربردها به‌طور مفصل خواهیم پرداخت.

پیش از LLMها از چه مدل‌هایی استفاده می‌شد؟

پیش از ظهور مدل‌های زبانی بزرگ، امبدینگ‌هایی مانند **word2vec**، **GloVe** و **fastText** بسیار رایج بودند. اگرچه امروز بیشتر با امبدینگ‌های زمینه‌مند جایگزین شده‌اند، اما هنوز در برخی حوزه‌ها (مانند سیستم‌های توصیه‌گر) کاربرد دارند.

الگوریتم **word2vec** بر دو ایده‌ی اصلی استوار است:

- **اسکیپ‌گرام (skip-gram)** برای انتخاب کلمات همسایه
- **نمونه‌گیری منفی (negative sampling)** برای ایجاد تمایز میان واژه‌های مرتبط و نامرتبط

این روش، نوعی آموزش مبتنی بر تضاد (**contrastive training**) محسوب می‌شود که در فصل‌های آخر به‌طور کامل بررسی خواهیم کرد.

امبدینگ‌ها در سیستم‌های توصیه‌گر

در ادامه دیدیم که چگونه می‌توان از امبدینگ‌ها برای ساخت سیستم‌های توصیه‌گر موسیقی استفاده کرد. با استفاده از الگوریتم **word2vec** و فهرست‌های پخش (**playlist**)، آهنگ‌ها را مانند واژه‌ها در جمله در نظر گرفتیم و بردارهایی ساختیم که شباهت میان آهنگ‌ها را به‌خوبی مدل‌سازی می‌کنند.

سایر

"Pasted image 20250917231730.png1" could not be found.

"Pasted image 20250917231801.png1" could not be found.

"Pasted image 20250917231833.png1" could not be found.

"Pasted image 20250917231858.png1" could not be found.

"Pasted image 20250917231915.png1" could not be found.

"Pasted image 20250917231938.png1" could not be found.

"Pasted image 20250917232010.png1" could not be found.

"Pasted image 20250917232047.png1" could not be found.

"Pasted image 20250917232104.png1" could not be found.

"Pasted image 20250917232125.png1" could not be found.

"Pasted image 20250917232136.png1" could not be found.

گام بعدی

در فصل آینده، به مرحله‌ی بعد از توکن‌سازی خواهیم پرداخت:
مدل زبانی بزرگ چگونه این توکن‌ها را پردازش می‌کند و متن تولید می‌کند؟

در آن فصل، مفاهیم اصلی پشت معماری ترنسفورمر را بررسی خواهیم کرد—ساختاری که قدرت خارق‌العاده مدل‌های زبانی مدرن از آن ناشی می‌شود.

"Pasted image 20251228174237.png1" could not be found.

"Pasted image 20251228174245.png1" could not be found.

"Pasted image 20251228174256.png1" could not be found.

"Pasted image 20251228174308.png1" could not be found.

"Pasted image 20251228174316.png1" could not be found.

"Pasted image 20251228174324.png1" could not be found.

"Pasted image 20251228174337.png1" could not be found.

"Pasted image 20251228174345.png1" could not be found.

"Pasted image 20251228174353.png1" could not be found.

"Pasted image 20251228174400.png1" could not be found.

"Pasted image 20251228174407.png1" could not be found.

"Pasted image 20251228174419.png1" could not be found.